

硕士学位论文

利用变分推断提升 MCMC 算法的效率

USING THE VARIATIONAL INFERENCE TO
IMPROVE THE EFFICIENCY OF MCMC
ALGORITHMS

研 究 生：殷萍萍

指 导 教 师：焦熙云助理教授

南方科技大学

二〇二四年五月

国内图书分类号：O212.8

国际图书分类号：519.2

学校代码：14325

密级：公开

理学硕士学位论文

利用变分推断提升 MCMC 算法的效率

学位申请人：殷萍萍

指导教师：焦熙云助理教授

学科名称：数学

答辩日期：2024 年 5 月

培养单位：统计与数据科学系

学位授予单位：南方科技大学

Classified Index: O212.8

U.D.C: 519.2

Thesis for the degree of Master of Science

USING THE VARIATIONAL INFERENCE TO IMPROVE THE EFFICIENCY OF MCMC ALGORITHMS

Candidate:	Yin Pingping
Supervisor:	Assistant Professor Jiao Xiyun
Speciality:	Mathematics
Date of Defence:	May, 2024
Affiliation:	Department of Statistics and Data Science
Degree-Confering-	Southern University of Science and
Institution:	Technology

学位论文公开评阅人和答辩委员会名单

公开评阅人名单

无（全匿名评阅）

答辩委员会名单

主席	史建清	教授	南方科技大学
委员	殷鹏	副研究员	华大基因
	徐匆	讲师	南方科技大学
秘书	王晓瑞	教辅序列	南方科技大学

摘要

贝叶斯统计基于贝叶斯定理进行推断，然而贝叶斯推断面临的巨大挑战是未知参数的后验分布通常没有明确的解析形式。实际中是用近似方法从数值上估计后验分布。两种常用的近似方法是马尔可夫链蒙特卡罗（Markov chain Monte Carlo, MCMC）和变分推断（variational inference, VI）。基于采样的 MCMC 方法能保证产生（渐进）精确的样本，但往往计算量很大。而基于优化的 VI 方法在计算速度和针对维度的扩展性方面有优势，但结果的准确性没有保证。本研究的目标是结合 MCMC 与 VI 两类方法，产生一类在性能上能同时兼具两者的新算法。

本研究提出的第一个新算法是利用具有不同协方差结构的高斯变分推断（Gaussian variational inference, GVI）来估计哈密顿蒙特卡罗（Hamiltonian Monte Carlo, HMC）算法中用于线性变换的预处理矩阵。该方法解决了在高维模型中计算预处理矩阵困难的问题。本文中对该方法的测试包括从维度和复杂性逐步增加的目标分布中进行采样。测试结果表明，使用 GVI 生成预处理矩阵的 HMC 在效率上优于标准的 MCMC 算法。随着目标分布的维度和复杂性增加，这种优势会变得更加明显。

第二个新提出的算法是将基于 VI 的生成模型-变分自动编码器（variational auto-encoder, VAE）和 Metropolis-Hastings（MH）算法相结合，结合的具体方法是使用 VAE 生成 MH 的提议分布。这种创新的混合方法称为“MH-VAE”算法，与其他现有的 MCMC 算法相比，它易于执行并且能更有效地探索整个参数空间。使用峰值相隔较远的多维混合高斯分布和多维混合 t 分布对该方法进行测试的结果表明，MH-VAE 算法总是能够成功地在多峰分布的所有峰值处采样，而 MH 算法经常卡在局部峰值处，导致无法收敛。

关键词：贝叶斯统计；效率；高斯变分推断；马尔科夫链蒙特卡罗；变分自动编码器

ABSTRACT

Bayesian statistics makes inference based on Bayes' theorem, and the biggest challenge faced by Bayesian inference is that the posterior distribution of the unknown parameters is typically analytically intractable. In practice, the posterior is estimated numerically using approximation methods, and two widely used such methods are Markov Chain Monte Carlo (MCMC) and variational inference (VI). Sampling-based MCMC methods can guarantee to produce (asymptotically) exact samples, but tend to be computationally intense, while optimizing-based VI methods are superior in speed and scalability but may suffer from inaccuracy. The goal of this study is to combine the MCMC and VI methods and produce a new class of algorithms that incorporate the strengths of both.

The first newly proposed algorithm is to utilize Gaussian variational inference (GVI) with various covariance structures to derive a preconditioning matrix for the linear transformation in Hamiltonian Monte Carlo (HMC). This method addresses the challenge of computing preconditioning matrices for high-dimensional models. The tests of this method include sampling from target distributions of increasing dimensions and complexity. The results of the tests show that the HMC with preconditioning matrix produced by GVI outperforms standard MCMC algorithms in efficiency. This superiority becomes more pronounced as the dimension and complexity of the target distribution increases.

The second newly proposed algorithm is to combine a VI-based generative model, variational auto-encoder (VAE), with the Metropolis–Hastings (MH) sampler, by using VAE to produce the proposal distribution of MH. This innovative hybrid method, termed the “VAE-MH” sampler, is straightforward to implement and more efficient in traversing the whole parameter space than state-of-the-art MCMC algorithms. The tests of this method using both multi-dimensional mixture Gaussian and mixture t distributions with distant modes show that the VAE-MH sampler can always successfully identify all the modes of a multi-modal distribution while the MH algorithm often gets trapped in local modes and fails to converge.

Keywords: Bayesian inference; Efficiency; Gaussian Variational Inference; Markov Chain Monte Carlo; Variational Auto–Encoder

TABLE OF CONTENTS

摘 要.....	I
ABSTRACT.....	II
CHAPTER 1 INTRODUCTION.....	1
1.1 Motivation.....	1
1.2 Contribution.....	2
1.3 Thesis Organization.....	3
CHAPTER 2 PRELIMINARIES	4
2.1 Markov Chain Monte Carlo	4
2.1.1 A brief introduction to Markov Chain.....	4
2.1.2 Markov Chain Monte Carlo algorithms.....	5
2.2 Variational Inference.....	6
2.2.1 A brief introduction to variational inference	6
2.2.2 Mean field variational inference	7
2.2.3 Fixed form variational inference	7
2.3 Generative Models	11
2.4 Related Work.....	12
2.4.1 MCMC and VI: bridging the gap	12
2.4.2 Using generative models to assist MCMC algorithms	13
CHAPTER 3 METHODOLOGY	14
3.1 State-of-the-art MCMC Algorithms	14
3.1.1 Random-walk Metropolis–Hastings algorithm	14
3.1.2 Adaptive Metropolis–Hastings algorithm.....	15
3.1.3 Metropolis-adjusted Langevin algorithm	15
3.1.4 Hamiltonian Monte Carlo algorithm	16
3.2 Estimation of Covariance Matrices	18
3.2.1 Estimate covariance matrix with burn-in sample	18
3.2.2 Gaussian variational inference	18
3.3 Linear Transformations in HMC via Gaussian Variational Inference	24

TABLE OF CONTENTS

3.4	Efficient Bayesian Sampling Using VAE to Assist MCMC Methods	26
3.4.1	A brief introduction to variational auto-encoder model	26
3.4.2	A VAE-assisted MCMC sampler	30
3.5	Performance Evaluation Criteria for Algorithms	32
CHAPTER 4	NUMERICAL STUDIES.....	34
4.1	Compare the Speed and Accuracy of Estimating Gaussian Target via MCMC and GVI.....	34
4.2	Linear Transformations in HMC via GVI.....	35
4.2.1	Posterior sampling in logistic regression models	35
4.2.2	Sampling from a 100-dimensional Gaussian distribution	38
4.2.3	Posterior sampling in a 509-dimensional GLMM	40
4.3	Efficient Bayesian Sampling Using the VAE-MH Algorithm	43
4.3.1	Sampling mixture of Gaussian and mixture of Student t	43
4.3.2	Sampling from 10-dimensional mixture Gaussian distribution.....	47
4.3.3	Sampling from 2-dimensional Gaussian mixture of 20 components.....	49
CONCLUSION.....		52
REFERENCES.....		54
ACKNOWLEDGEMENTS.....		57
RESUME AND ACADEMIC ACHIEVEMENTS		58

CHAPTER 1 INTRODUCTION

1.1 Motivation

Bayesian statistics makes inference based on Bayes' theorem. Specially, consider the model $p(\mathbf{x}|\boldsymbol{\theta})$, where $\mathbf{x}$ is the data, and $\boldsymbol{\theta}$ is the unknown parameter in the model. Given the prior distribution $p(\boldsymbol{\theta})$, by Bayes's theorem, we obtain the posterior distribution of $\boldsymbol{\theta}$,

$$p(\boldsymbol{\theta}|\mathbf{x}) = \frac{p(\boldsymbol{\theta})p(\mathbf{x}|\boldsymbol{\theta})}{p(\mathbf{x})} = \frac{p(\boldsymbol{\theta})p(\mathbf{x}|\boldsymbol{\theta})}{\int p(\boldsymbol{\theta})p(\mathbf{x}|\boldsymbol{\theta})d\boldsymbol{\theta}}.$$

However the integral in the denominator is not feasible in practical scenarios. Since the posterior distribution plays the central role in Bayesian analysis, approximating the posterior without doing the integral is the main task of Bayesian inference. Markov Chain Monte Carlo (MCMC) and variational inference (VI) are two primary approaches to estimating posterior distributions. Both approaches have their advantages and disadvantages.

MCMC has been the gold standard for sampling from posterior distributions since it can guarantee sampling from the exact target distribution. The Metropolis–Hastings (MH)^[1-2] is one of the most popular MCMC methods. MH generates samples from the target distribution by proposing candidates from a proposal distribution and accepting them with a probability. The random-walk MH algorithm is particularly attractive due to its generality and ease of implementation. However, the proposal distribution of a random-walk MH only generates local moves, which travel to the high-density region by chance. If the dimension of the target distribution is high, the random-walk MH has low acceptance rate, highly correlated samples and its efficiency decreases quickly^[3]. In addition, a well-known challenge for standard MCMC methods is their poor mixing when dealing with multi-modal target distributions with distant modes. Many MCMC algorithms struggle to jump over the low-probability region between modes, and may get stuck at one mode.

VI is an optimization-based approach to approximating distributions^[4-6]. In contrast to sampling-based MCMC approaches, VI searches for the optimal distribution that minimizes the distance (typically measured by the Kullback–Leibler (KL) divergence) to the target posterior from a set of tractable distributions. Compared to MCMC, VI is much less expensive in computation and much more scalable with dimension^[7]. However, VI can introduce significant bias if all the distributions in the candidate family are far away from the target distribution. Thus, the effect of VI highly depends on the choice of the

candidate distributions^[8].

As stated above, VI and MCMC methods have complementary strengths and weaknesses. Therefore, it is natural to expect that new algorithms with performance better than both can be obtained by combining VI and MCMC. However, simply using the VI approximation as the proposal distribution of an MH sampler lead to algorithms with poor scalability and mixing efficiency^[9-11]. In this study, we present two novel approaches to combining VI and MCMC, which can truly utilize the strengths of both to generate more efficient algorithms.

1.2 Contribution

The goal of this study is to combine the MCMC with VI methods and produces a class of algorithms that incorporate the strengths of both.

The first newly proposed algorithm is to utilize Gaussian variational inference (GVI) with various covariance structures to derive a preconditioning matrix for the linear transformation in HMC (Hamiltonian Monte Carlo). Specifically, we consider GVI with Cholesky decomposed covariance^[12], GVI with factor decomposed covariance^[13] and GVI with sparse precision matrices^[14]. This method addresses the challenge of computing preconditioning matrices for high-dimensional models^[3]. The tests of this method in this thesis include sampling from target distributions of increasing dimensions and complexity. The results of the tests show that the HMC with preconditioning matrix produced by GVI outperforms standard MCMC algorithms in efficiency. This superiority becomes more pronounced as the dimension and complexity of the target distribution increases.

The second newly proposed algorithm is to combine VI-based generative model, variational auto-encoder (VAE)^[15], with MH^[2] sampler, by using VAE to produce the proposal distribution of MH. This innovative hybrid method, termed the “VAE-MH” sampler, is straightforward to implement and more efficient in traversing the whole parameter space than state-of-the-art MCMC algorithms. The tests of the method using both multi-dimensional mixture Gaussian and mixture t distributions with distant modes show that the VAE-MH sampler can always successfully identify all the modes of a multi-modal distribution while the MH algorithm often gets trapped in local modes and fails to converge.

1.3 Thesis Organization

The structure of the subsequent chapters is as follows. Chapter 2 includes an overview of MCMC algorithms, VI, and generative models. Furthermore, some existing methods which show better performance by combining MCMC with VI or generative models are also introduced. Chapter 3 introduces the state-of-the-art MCMC algorithms which are used in the numerical studies of Chapter 4, outlines the new methods that are proposed in this thesis, describes the technique of tuning different algorithms and also the criteria used to compare different methods. Chapter 4 contains the numerical studies, and finally a summary and some discussions are provided in Conclusion.

CHAPTER 2 PRELIMINARIES

This chapter contains a literature review of the MCMC algorithms, VI, and generative models, and also introduces the existing algorithms which exhibit good performance by combining these methods.

2.1 Markov Chain Monte Carlo

2.1.1 A brief introduction to Markov Chain

The MCMC algorithm is named so because the samples generated by it form a Markov chain. A sequence of random variables, i.e. , $\boldsymbol{\theta}^{(0)}, \boldsymbol{\theta}^{(1)}, \dots, \boldsymbol{\theta}^{(T)} \dots$ is a Markov chain if

$$\mathbb{P}(\boldsymbol{\theta}^{(t+1)} \in A | \boldsymbol{\theta}^{(0)}, \dots, \boldsymbol{\theta}^{(t)}) = \mathbb{P}(\boldsymbol{\theta}^{(t+1)} \in A | \boldsymbol{\theta}^{(t)}),$$

for each t , where A is a measurable set in the support of $\boldsymbol{\theta}$.

An MCMC algorithm is characterized by its transition kernel, that is, $K(\boldsymbol{\theta}^{(t+1)} | \boldsymbol{\theta}^{(t)})$, which is designed to have the target distribution $p(\boldsymbol{\theta})$ as the stationary distribution of the corresponding Markov chain, i.e. ,

$$p(\boldsymbol{\theta}') = \int K(\boldsymbol{\theta}' | \boldsymbol{\theta}) p(\boldsymbol{\theta}) d\boldsymbol{\theta}.$$

When converging, MCMC uses the sample mean to approximate the true expectation, i.e.,

$$\int g(\boldsymbol{\theta}) p(\boldsymbol{\theta}) d\boldsymbol{\theta} \approx \frac{1}{T} \sum_{t=1}^T g(\boldsymbol{\theta}^{(t)}),$$

where g is a p -integrable function of $\boldsymbol{\theta}$. This is why MCMC is one of the Monte Carlo methods.

A Markov chain having the following properties can guarantee to have a unique stationary distribution and also converge to it^[7,16].

- (1) Aperiodicity : the chain does not has any deterministic cycle.
- (2) Irreducibility: the chain does not “get stuck” in a local region of the sample space.
- (3) Harris recurrence: the chain visits any region of the sample space “sufficiently often”.

Fortunately, the Markov chains produced by most MCMC algorithms own the three properties. For such a Markov chain, the sample mean $\frac{1}{T} \sum_{t=1}^T g(\boldsymbol{\theta}^{(t)})$ also converges to the true expectation $\int g(\boldsymbol{\theta})p(\boldsymbol{\theta})d\boldsymbol{\theta}$ almost surely.

2.1.2 Markov Chain Monte Carlo algorithms

As stated in Chapter 1, MH is one of the most widely used MCMC methods. The Metropolis algorithm (1953)^[1,17] was first proposed, and Hasting et al. (1970)^[2] generalized the Metropolis algorithm by allowing the asymmetric in the proposal distribution. The Gibbs sampler^[18] is another widely used MCMC algorithms. It samples from the target distribution by drawing from the conditional distribution of each component variable. Each step of a Gibbs sampler can be considered a special MH algorithm with acceptance probability 1. While the Gibbs sampler is easy to implement when sampling from a multi-dimensional distribution, it has been criticized for slow convergence. In this thesis, we focus on the MH-type MCMC algorithms.

A common choice for the proposal distribution in MH is a Gaussian or uniform distribution around the current state of the Markov chain. The corresponding MH algorithm is also called random walk. The random-walk MH algorithm is easy to implement, but does not have good mixing properties or scale well with dimension, because the moves it proposes are local and random.

Many studies for improving the efficiency of the MH algorithm have been concentrated on choosing better proposal distributions. Yang et al. (2013)^[19] introduced a novel class of Bactrian kernels, which are able to propose samples distant from the current state. Based on Yang et al. (2013), Thawornwattana et al. (2018)^[20] introduced the “mirror” proposal which can lead to Markov chains with negative autocorrelations and super efficiency. Some MH algorithms incorporate the gradient information of the target distribution in the proposal. Both the Metropolis adjusted Langevin algorithm (MALA)^[3,21] and Hamiltonian Monte Carlo (HMC)^[22] are such MH methods. MALA derives the proposal from a discretized Langevin diffusion and HMC uses discretized Hamiltonian dynamics to propose new values. Both methods are more efficient and scalable than random-walk MH. However, tuning the parameters of these methods is a challenge. To address this problem, Girolami et al. (2011)^[3] introduced Riemann-manifold MALA and HMC, which designs MCMC algorithms on the Riemann manifold defined by the target distribution to account for its geometric structure. However, Riemann-manifold MCMC algorithm are expensive in computation. Hoffman et al. (2014)^[23] proposed the No-U-Turn samplers (NUTS), an

extension of HMC that eliminates the necessity of manually tuning the algorithm.

In addition, some adaptive MCMC algorithms are invented. For example, Haario et al. (2001)^[24] introduced the adaptive Metropolis (AM) algorithm, which automatically adjusts the covariance matrix of proposal distribution to achieve a desired acceptance rate of the MH algorithm.

Multi-modal target distributions impose big challenges on the efficiency of the standard MCMC algorithms since the latter typically have trouble jumping over low-density areas between modes and will get stuck. Several strategies have been proposed to address this challenge. One widely used technique is tempering, which heated the multi-modal target distribution to flatten the modes. Specifically, the target distribution is raised by a power $\beta \in (0, 1)$ and samples from the flattened distribution $p(\theta)^\beta$ are facilitated to help traverse between modes when sampling from the un-flattened target. Examples include parallel tempering^[25] and simulated tempering^[26]. Other approaches for dealing with multi-modal targets include designing special proposal distributions tailored to facilitate movement between modes^[27].

2.2 Variational Inference

2.2.1 A brief introduction to variational inference

Variational inference is a common approach for scalable approximation in Bayesian inference, particularly when dealing with large datasets and highly parameterized models. This section provides a concise overview of VI. For a more comprehensive review, refer to Blei et al. (2017)^[6] and Tran et al. (2021)^[12].

To approximate the posterior distribution $p(\theta|\mathbf{x})$ in Bayesian inference, VI searches for the optimal distribution in a distribution family $Q = \{q_\lambda(\theta)\}$ parameterized by λ . The optimal distribution is closest to the exact posterior in terms of the KL divergence, that is,

$$q_\lambda^*(\theta) = \arg \min_{q_\lambda(\theta) \in Q} \text{KL}(q_\lambda(\theta) || p(\theta|\mathbf{x})).$$

The KL divergence between $q_\lambda(\theta)$ and $p(\theta|\mathbf{x})$ is

$$\begin{aligned} \text{KL}(q_\lambda(\theta) || p(\theta|\mathbf{x})) &= \int \log \frac{q_\lambda(\theta)}{p(\theta|\mathbf{x})} q_\lambda(\theta) d\theta \\ &= \mathbb{E}_{q_\lambda}[\log q_\lambda(\theta)] - \mathbb{E}_{q_\lambda}[\log p(\theta|\mathbf{x})] \\ &= \mathbb{E}_{q_\lambda}[\log q_\lambda(\theta)] - \mathbb{E}_{q_\lambda}[\log p(\theta, \mathbf{x})] + \log p(\mathbf{x}), \end{aligned}$$

where the $p(\mathbf{x})$ is the marginal likelihood. The KL divergence is asymmetric and always

non-negative. However, direct computation of the KL divergence is typically impractical due to the intractability of $p(\mathbf{x})$. Thus an alternative objective function called the evidence lower bound (ELBO) is adopted. The definition of the ELBO is,

$$\text{ELBO}(q_\lambda) = \mathbb{E}_{q_\lambda}[\log p(\boldsymbol{\theta}, \mathbf{x})] - \mathbb{E}_{q_\lambda}[\log q_\lambda(\boldsymbol{\theta})].$$

The relationship between marginal likelihood, KL divergence, and ELBO is shown in the following formula,

$$\log p(\mathbf{x}) = \text{KL}(q_\lambda(\boldsymbol{\theta}) || p(\boldsymbol{\theta} | \mathbf{x})) + \text{ELBO}(q_\lambda),$$

and thus ELBO is a lower bound of $\log p(\mathbf{x})$ due to the fact that the KL divergence is non-negative, and by maximizing the ELBO, we effectively minimize the KL divergence.

VI algorithms can be roughly divided into two main categories, mean-field VI (MFVI) and fixed-form VI (FFVI)^[12], which are introduced in the next section.

2.2.2 Mean field variational inference

For MFVI, we assume that the components of $\boldsymbol{\theta}$ are independent and each is governed by a separate density in the variational distribution, i.e.,

$$q_\lambda(\boldsymbol{\theta}) = \prod_{i=1}^d q_{\lambda_i}(\boldsymbol{\theta}_i).$$

For MFVI, the optimal $q_\lambda^*(\boldsymbol{\theta})$ can be found by iteratively updating each factor of the variational distribution $q_\lambda(\boldsymbol{\theta})$ using coordinate ascent variational inference (CAVI)^[6,28]. A common choice for $q_{\lambda_i}(\boldsymbol{\theta}_i)$ can be Gaussian distribution.

Although MFVI is easy to implement and typically provides accurate estimates of the location of the target distribution, it may yield poor estimates of high-order moments and miss the correlation between parameters.

2.2.3 Fixed form variational inference

2.2.3.1 Stochastic gradient estimators

In FFVI, we assume a fixed parametric form for the variational distribution q , i.e. $q_\lambda \in Q$, where Q is some tractable distribution family. As stated in section 2.2.1, the optimal variational distribution q^* is obtained by maximizing the ELBO, i.e.,

$$\text{LB}(\lambda) := \text{ELBO}(q_\lambda) = \mathbb{E}_{q_\lambda} \left[\log \frac{p(\boldsymbol{\theta})p(\mathbf{x}|\boldsymbol{\theta})}{q_\lambda(\boldsymbol{\theta})} \right] = \mathbb{E}_{q_\lambda} [h_\lambda(\boldsymbol{\theta})], \quad (2-1)$$

where

$$h_{\lambda}(\boldsymbol{\theta}) := \log \left[\frac{p(\boldsymbol{\theta})p(\mathbf{x}|\boldsymbol{\theta})}{q_{\lambda}(\boldsymbol{\theta})} \right]. \quad (2-2)$$

While some trivial cases allow analytical computation of the evidence lower bound, in most situations, stochastic optimization techniques are employed to maximize $\text{LB}(\boldsymbol{\lambda})$. The gradient of the ELBO is

$$\begin{aligned} \nabla_{\lambda} \text{LB}(\boldsymbol{\lambda}) &= \int_{\boldsymbol{\theta}} \nabla_{\lambda} q_{\lambda}(\boldsymbol{\theta}) \log \frac{p(\boldsymbol{\theta})p(\mathbf{x}|\boldsymbol{\theta})}{q_{\lambda}(\boldsymbol{\theta})} d\boldsymbol{\theta} - \int_{\boldsymbol{\theta}} q_{\lambda}(\boldsymbol{\theta}) \nabla_{\lambda} \log q_{\lambda}(\boldsymbol{\theta}) d\boldsymbol{\theta} \\ &= \int_{\boldsymbol{\theta}} q_{\lambda}(\boldsymbol{\theta}) \nabla_{\lambda} \log q_{\lambda}(\boldsymbol{\theta}) \log \frac{p(\boldsymbol{\theta})p(\mathbf{x}|\boldsymbol{\theta})}{q_{\lambda}(\boldsymbol{\theta})} d\boldsymbol{\theta} - \int_{\boldsymbol{\theta}} \nabla_{\lambda} q_{\lambda}(\boldsymbol{\theta}) d\boldsymbol{\theta} \\ &= \int_{\boldsymbol{\theta}} q_{\lambda}(\boldsymbol{\theta}) \nabla_{\lambda} \log q_{\lambda}(\boldsymbol{\theta}) \log \frac{p(\boldsymbol{\theta})p(\mathbf{x}|\boldsymbol{\theta})}{q_{\lambda}(\boldsymbol{\theta})} d\boldsymbol{\theta} - \nabla_{\lambda} \int_{\boldsymbol{\theta}} q_{\lambda}(\boldsymbol{\theta}) d\boldsymbol{\theta} \\ &= \mathbb{E}_{q_{\lambda}} \left[\nabla_{\lambda} \log q_{\lambda}(\boldsymbol{\theta}) \times \log \frac{p(\boldsymbol{\theta})p(\mathbf{x}|\boldsymbol{\theta})}{q_{\lambda}(\boldsymbol{\theta})} \right] \\ &= \mathbb{E}_{q_{\lambda}} [\nabla_{\lambda} \log q_{\lambda}(\boldsymbol{\theta}) \times h_{\lambda}(\boldsymbol{\theta})], \end{aligned} \quad (2-3)$$

and FFVI typically employs stochastic gradient ascent (SGA) to maximize the ELBO. However, the naive gradient estimator $\frac{1}{S} \sum_{s=1}^S \nabla_{\lambda} \log q_{\lambda}(\boldsymbol{\theta}_s) \times h_{\lambda}(\boldsymbol{\theta}_s)$ where $\boldsymbol{\theta}_1, \dots, \boldsymbol{\theta}_S$ sample from $q_{\lambda}(\boldsymbol{\theta})$, often exhibit significant variance. This variance arises primarily due to the fact that the logarithmic term “ $\log q_{\lambda}(\boldsymbol{\theta})$ ” tends to amplify small changes. Fortunately, techniques for reducing the variance of gradient estimators have been introduced, including control variates^[29-31] and the reparameterization trick^[15]. In this thesis, we use the reparameterization trick to reduce variance due to its simplicity and effectiveness.

2.2.3.2 Reparameterization trick

The reparameterization trick^[15] reduces the variance of gradient estimator by transforming the random variable $\boldsymbol{\theta}$ to a deterministic function of a standard random variable $\boldsymbol{\epsilon}$, whose distribution is known. Specifically, $\boldsymbol{\theta} = g(\boldsymbol{\lambda}, \boldsymbol{\epsilon})$, with $\boldsymbol{\epsilon}$ a sample from the distribution $p_{\boldsymbol{\epsilon}}(\cdot)$ and $\boldsymbol{\theta} \sim q_{\lambda}(\cdot)$. For example, if $q_{\lambda}(\boldsymbol{\theta})$ follows a normal distribution $N(\boldsymbol{\theta}|\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$, then we can reparameterize $\boldsymbol{\theta}$ as $\boldsymbol{\theta} = \boldsymbol{\mu} + \boldsymbol{\sigma}\boldsymbol{\epsilon}$, where $\boldsymbol{\epsilon} \sim N(0, 1)$. Then $\text{LB}(\boldsymbol{\lambda})$ becomes

$$\text{LB}(\boldsymbol{\lambda}) = \mathbb{E}_{\boldsymbol{\epsilon} \sim p_{\boldsymbol{\epsilon}}} [h_{\lambda}(g(\boldsymbol{\epsilon}, \boldsymbol{\lambda}))], \quad (2-4)$$

where $\mathbb{E}_{\boldsymbol{\epsilon} \sim p_{\boldsymbol{\epsilon}}}(\cdot)$ denotes the expectation with respect to $p_{\boldsymbol{\epsilon}}(\cdot)$. The derivative of $\text{LB}(\boldsymbol{\lambda})$ with respect to $\boldsymbol{\lambda}$ becomes

$$\nabla_{\lambda} \text{LB}(\boldsymbol{\lambda}) = \mathbb{E}_{\boldsymbol{\epsilon} \sim p_{\boldsymbol{\epsilon}}} [\nabla_{\lambda} g(\boldsymbol{\lambda}, \boldsymbol{\epsilon})^{\top} \nabla_{\boldsymbol{\theta}} h_{\lambda}(\boldsymbol{\theta})] + \mathbb{E}_{\boldsymbol{\epsilon} \sim p_{\boldsymbol{\epsilon}}} [\nabla_{\lambda} h_{\lambda}(\boldsymbol{\theta})],$$

where $\boldsymbol{\theta} = g(\boldsymbol{\epsilon}, \boldsymbol{\lambda})$, and $\nabla_{\boldsymbol{\lambda}} g(\boldsymbol{\lambda}, \boldsymbol{\epsilon})$ denotes the Jacobin matrix of dimensions $d_{\boldsymbol{\theta}} \times d_{\boldsymbol{\lambda}}$ corresponding to the vector-valued function $\boldsymbol{\theta} = g(\boldsymbol{\lambda}, \boldsymbol{\epsilon})$. Note that by manipulating the expectation term, we find that

$$\begin{aligned} \mathbb{E}_{\boldsymbol{\epsilon} \sim p_{\boldsymbol{\epsilon}}} (\nabla_{\boldsymbol{\lambda}} h_{\boldsymbol{\lambda}}(\boldsymbol{\theta})) &= \mathbb{E}_{\boldsymbol{\epsilon} \sim p_{\boldsymbol{\epsilon}}} [\nabla_{\boldsymbol{\lambda}} h_{\boldsymbol{\lambda}}(\boldsymbol{\theta} = g(\boldsymbol{\epsilon}, \boldsymbol{\lambda}))] \\ &= -\mathbb{E}_{\boldsymbol{\epsilon} \sim p_{\boldsymbol{\epsilon}}} [\nabla_{\boldsymbol{\lambda}} \log q_{\boldsymbol{\lambda}}(\boldsymbol{\theta} = g(\boldsymbol{\epsilon}, \boldsymbol{\lambda}))] \\ &= -\mathbb{E}_{\boldsymbol{\theta} \sim q_{\boldsymbol{\lambda}}} [\nabla_{\boldsymbol{\lambda}} \log q_{\boldsymbol{\lambda}}(\boldsymbol{\theta})] = 0. \end{aligned}$$

Therefore,

$$\nabla_{\boldsymbol{\lambda}} \text{LB}(\boldsymbol{\lambda}) = \mathbb{E}_{\boldsymbol{\epsilon} \sim p_{\boldsymbol{\epsilon}}} [\nabla_{\boldsymbol{\lambda}} g(\boldsymbol{\lambda}, \boldsymbol{\epsilon})^{\top} \nabla_{\boldsymbol{\theta}} h_{\boldsymbol{\lambda}}(\boldsymbol{\theta})]. \quad (2-5)$$

Then the unbiased estimate of the gradient can be

$$\nabla_{\boldsymbol{\lambda}} \widehat{\text{LB}}(\boldsymbol{\lambda}) = \frac{1}{S} \sum_{s=1}^S \nabla_{\boldsymbol{\lambda}} g(\boldsymbol{\lambda}, \boldsymbol{\epsilon}_s)^{\top} \nabla_{\boldsymbol{\theta}} \{h_{\boldsymbol{\lambda}}[g(\boldsymbol{\lambda}, \boldsymbol{\epsilon}_s)]\},$$

where $\boldsymbol{\epsilon}_s \sim p_{\boldsymbol{\epsilon}}(\cdot)$, $s = 1, \dots, S$.

2.2.3.3 Stochastic gradient variational inference

For the objective function $\text{LB}(\boldsymbol{\lambda})$, $\nabla_{\boldsymbol{\lambda}} \text{LB}(\boldsymbol{\lambda})$ is its gradient, and $\nabla_{\boldsymbol{\lambda}} \widehat{\text{LB}}(\boldsymbol{\lambda})$ is the unbiased estimator. The SGA^[13] algorithm updates the variational parameters at each iteration t by

$$\boldsymbol{\lambda}^{(t+1)} = \boldsymbol{\lambda}^{(t)} + \boldsymbol{\rho}_t \odot \nabla_{\boldsymbol{\lambda}} \widehat{\text{LB}}(\boldsymbol{\lambda}^{(t)}),$$

where the symbol $\odot$ denotes the Hadamard product of two random vectors, and $\boldsymbol{\rho}_t$ represent the learning-rate vector, which can change with t . When training complicated models, SGA can get stuck in local minima if the learning rate is too small or fail to converge if the learning rate is too large. To address these problems, it is advised to use adaptive learning rates. Below is a simple adaptive learning-rate algorithm.

Let $\mathbf{g}_t := \nabla_{\boldsymbol{\lambda}} \widehat{\text{LB}}(\boldsymbol{\lambda}^{(t)})$ and $\mathbf{v}_t := (\mathbf{g}_t)^2$ where the square is a coordinate-wise operator, and then

(1) initialize $\mathbf{g}_0$, $\mathbf{v}_0$ and $\boldsymbol{\lambda}^{(0)}$, and set $\bar{\mathbf{g}} = \mathbf{g}_0$, $\bar{\mathbf{v}} = \mathbf{v}_0$. Set adaptive learning weights $\beta_1, \beta_2 \in (0, 1)$. In our numerical studies, we choose $\beta_1 = 0.9, \beta_2 = 0.9$.

(2) for $t = 0, 1, \dots, T$, do

$$\bar{\mathbf{g}} = \beta_1 \bar{\mathbf{g}} + (1 - \beta_1) \mathbf{g}_t,$$

$$\bar{\mathbf{v}} = \beta_2 \bar{\mathbf{v}} + (1 - \beta_2) \mathbf{v}_t,$$

$$\boldsymbol{\lambda}^{(t+1)} = \boldsymbol{\lambda}^{(t)} + \rho_t \bar{\mathbf{g}} / \sqrt{\bar{\mathbf{v}}},$$

where ρ_t is a scalar defined by:

$$\rho_t = \min\left(\epsilon_0, \epsilon_0 \frac{\tau}{t}\right) = \begin{cases} \epsilon_0, & t \leq \tau, \\ \epsilon_0 \frac{\tau}{t}, & t > \tau, \end{cases}$$

with ϵ_0 a small fixed value (e.g., 0.1 or 0.01) and τ a threshold for the number of iterations. When $t \geq \tau$, the learning rate starts decreasing^[12].

Furthermore, the ADADELTA technique^[32], which is a more advanced method for adapting learning rates, was utilized in our study. Specifically, the i th element of $\boldsymbol{\lambda}$, λ_i is updated at iteration $t + 1$ as

$$\lambda_i^{(t+1)} = \lambda_i^{(t)} + \Delta \lambda_i^{(t)}.$$

In this equation, the increment $\Delta \lambda_i^{(t)} = \rho_i^{(t)} g_{\lambda_i}^{(t)}$, where $g_{\lambda_i}^{(t)}$ represents the i th component of the estimated gradient $\nabla_{\boldsymbol{\lambda}} \widehat{\text{LB}}(\boldsymbol{\lambda}^{(t)})$, and $\rho_i^{(t)}$ is

$$\rho_i^{(t)} = \frac{\sqrt{\mathbb{E}(\Delta \lambda_i^2)^{(t-1)} + \delta}}{\sqrt{\mathbb{E}(g_{\lambda_i}^2)^{(t)} + \delta}}.$$

Here, δ is a very small positive constant, $\mathbb{E}(\Delta \lambda_i^2)^{(t-1)}$ and $\mathbb{E}(g_{\lambda_i}^2)^{(t)}$ are the decaying averages of $\Delta \lambda_i^{(t)^2}$ and $g_{\lambda_i}^{(t)^2}$, respectively, that is^[13]

$$\mathbb{E}(\Delta \lambda_i^2)^{(t)} = v \mathbb{E}(\Delta \lambda_i^2)^{(t-1)} + (1 - v) \Delta \lambda_i^{(t)^2},$$

$$\mathbb{E}(g_{\lambda_i}^2)^{(t)} = v \mathbb{E}(g_{\lambda_i}^2)^{(t-1)} + (1 - v) g_{\lambda_i}^{(t)^2},$$

where $0 < v < 1$ represents a decaying constant. In our numerical studies using the GVI with factor decomposed covariance and GVI with sparse precision matrices, we set $\delta = 10^{-6}$, $v = 0.95$ and $\mathbb{E}(\Delta \lambda_i^2)^{(0)} = \mathbb{E}(g_{\lambda_i}^2)^{(0)} = 0$.

2.2.3.4 Stopping rule

The unbiased estimate of the objective function defined in equation (2-1) is

$$\widehat{\text{LB}}(\boldsymbol{\lambda}) = \frac{1}{S} \sum_{s=1}^S h_{\boldsymbol{\lambda}}(\boldsymbol{\theta}_s), \boldsymbol{\theta}_s \sim q_{\boldsymbol{\lambda}}(\boldsymbol{\theta}),$$

where $h_{\boldsymbol{\lambda}}(\boldsymbol{\theta})$ is defined in equation (2-2).

We generally employ a moving average of $\widehat{\text{LB}}(\boldsymbol{\lambda})$ over a window of t_w iterations to address the sampling noise in $\widehat{\text{LB}}(\boldsymbol{\lambda})$ ^[12]. Denote the moving average of $\widehat{\text{LB}}(\boldsymbol{\lambda})$ by

$\overline{\text{LB}}(\lambda^{(t)})$ and compute it as

$$\overline{\text{LB}}(\lambda^{(t)}) = \frac{1}{t_W} \sum_{k=1}^{t_W} \widehat{\text{LB}}(\lambda^{(t-k+1)}).$$

We keep a record of the maximum $\overline{\text{LB}}(\lambda^{(t)})$ so far, denote it as by $\overline{\text{LB}}_{\max}$, and we stop the VI algorithm if $\overline{\text{LB}}(\lambda^{(t)})$ falls below $\overline{\text{LB}}_{\max}$ for P consecutive times, where P is called the “patience parameter”^[12]. Typical choices for t_W and P are 20 or 50^[12].

2.3 Generative Models

Generative models are designed to learn the unknown data distribution using the data from that distribution and generate new data that closely resemble the real data. Mathe-

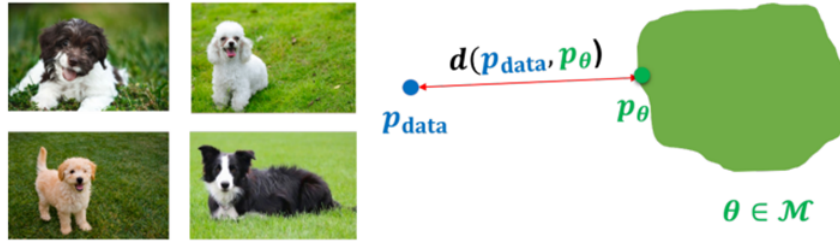


Figure 2-1 Generative model learning

matically, the task of generative model is to find a distribution that is closest to the true data distribution p_{data} from a set of candidate distributions parameterized by θ , i.e.,

$$\min_{\theta \in \mathcal{M}} d(p_{\text{data}}, p_{\theta}),$$

where $d(\cdot)$ represents a measure of distance between probability distributions. In our study, we use the new sample drawn by the generative model to produce a proposal distribution for MH.

Popular generative models are Generative Adversarial Networks (GAN), Variational Auto-encoder (VAE), Normalizing flow and diffusion models. See figure 2-2, GAN^[33] consists of two parts, a generator and a discriminator. Both parts can be neural networks. These two parts participate in a competitive game where the generator generates synthetic data to fool the discriminator, whose task is then to distinguish real and synthetic data. The generator and discriminator are trained simultaneously. If the global equilibrium is reached, the distribution of both the generator and the discriminator are equal to the true data distribution. Normalizing flow^[34] is a generative model designed to convert a simple distribution such as the standard normal distribution, into a complex distribution equal to

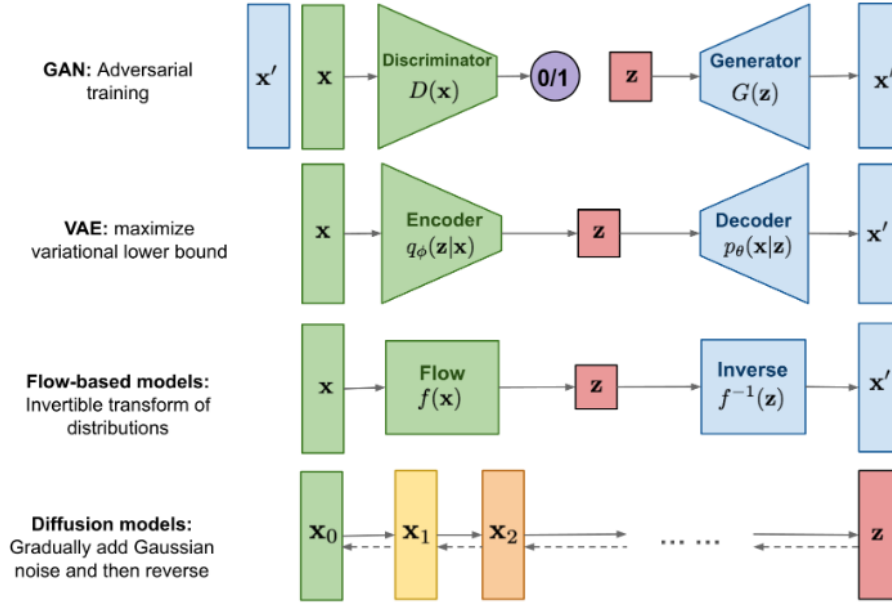


Figure 2-2 Overview of different types of generative models

or very close to the data distribution. This is achieved by a sequence of invertible and differentiable transformations. Inspired by non-equilibrium thermodynamics, diffusion models^[35-36] first use a forward process to add noise to the data until the data becomes pure noise. Then diffusion models learn the reverse of the forward process and use the reverse process to recover the data distribution. VAE^[15] assumes that the data are generated by first drawing a latent variable z from its prior distribution $p(z)$ and then drawing x from $p_\theta(x|z)$ given z . However, the generating process is mostly hidden. Therefore, VAE first obtains an approximation $q_\lambda(z|x)$ of the posterior distribution of z given x , i.e. $p_\theta(z|x)$. Then VAE treats $q_\lambda(z|x)$ as the encoder, sample the code z from it and given this code, sample new data x from $p_\theta(x|z)$, which is in fact the decoder. Since VAE is closely related to VI, it is the generative model that we consider when designing new algorithms. A detailed introduction of VAE is in section 3.4.1.

2.4 Related Work

2.4.1 MCMC and VI: bridging the gap

A logical inquiry emerges: is it feasible to merge MCMC and VI to alleviate their individual shortcomings and harness the strengths of both? Some attempts have already been made in this direction. De Freitas et al. (2001)^[11] introduced an approach which constructs a new blocked MH algorithm using the variational approximation as the proposal distribution, and switches between the new MH and a local MH algorithm. This

strategy allows the quick identification of high-density regions, and at the same time avoids getting stuck. Salimans et al. (2015)^[37] introduced another approach that combines VI and MCMC. They used a few samples of MCMC as auxiliary variables in the variational approximation distribution to increase the flexibility of the variational distribution family. Another approach incorporating auxiliary variables, called auxiliary variational MCMC, was proposed by Habib et al. (2018)^[38]. This method introduces auxiliary variables to construct a joint distribution with the target distribution as the marginal. VI is then performed to approximate this joint distribution and finally a new MCMC algorithm sampling the joint distribution is constructed using the variational approximation as the proposal. This new MCMC is able to avoid the random-walk behavior often encountered in traditional MCMC methods and enables more efficient sampling.

2.4.2 Using generative models to assist MCMC algorithms

The research in this direction so far has mostly been using the generative model to construct the proposal distribution of the MH algorithm. Gabriele et al. (2021)^[39] used the MALA algorithm to train the normalizing flow and used the trained normalizing flow to construct the proposal of a new MH algorithm. Hunt-Smith et al. (2024)^[40] used the random-walk MH to train the diffusion model and then the diffusion model is used to construct the proposal distribution for a new MH algorithm. The new MH algorithms of both Gabriele et al. (2021) and Hunt-Smith et al. (2024) are efficient in sampling multi-modal distributions.

CHAPTER 3 METHODOLOGY

The state of the art MCMC algorithms we will use to develop new algorithms are introduced in Section 3.1 and two methods for estimating the covariance matrix are discussed in Section 3.2. Two new algorithms which combine VI and MCMC algorithms that we proposed are introduced in section 3.3 and section 3.4 respectively. The first new algorithm is to utilize GVI with various covariance structures to derive a preconditioning matrix for the linear transformation in HMC. This method addresses the challenge of computing preconditioning matrices for high-dimensional models. The second new algorithm is to combine a VI-based generative model, VAE, with the MH sampler, by using VAE to produce the proposal distribution of MH. This innovative hybrid method, termed the “MH-VAE” sampler, is straightforward to implement and more efficient in traversing the whole parameter space than state-of-the-art MCMC algorithms. In Section 3.5, the optimal stepsize of the commonly used MCMC algorithms and the criteria to assess the performance of algorithms are introduced.

3.1 State-of-the-art MCMC Algorithms

3.1.1 Random-walk Metropolis–Hastings algorithm

Given the target distribution $p(\cdot)$ and a proposal distribution Q , each iteration of the MH algorithm proceeds as

- (1) generate $\theta^* \sim Q(\theta|\theta^{(t)})$,
- (2) calculate the MH acceptance probability,

$$\alpha(\theta^{(t)}, \theta^*) = \min \left\{ \frac{p(\theta^*)}{p(\theta^{(t)})} \frac{Q(\theta^{(t)} | \theta^*)}{Q(\theta^* | \theta^{(t)})}, 1 \right\},$$

- (3) let

$$\theta^{(t+1)} = \begin{cases} \theta^*, & \text{with probability } \alpha(\theta^{(t)}, \theta^*), \\ \theta^{(t)}, & \text{with probability } 1 - \alpha(\theta^{(t)}, \theta^*). \end{cases}$$

As stated in section 2.1.2, for random-walk MH, the proposal distribution is a Gaussian or Uniform centered around the current state $\theta^{(t)}$. In multi-dimensional cases, the uniform kernel can take the form of either a cube or a sphere. Therefore, the Hastings ratio $\frac{Q(\theta^{(t)}|\theta^*)}{Q(\theta^*|\theta^{(t)})}$

becomes 1, and thus random-walk MH is in fact a Metropolis algorithm.

3.1.2 Adaptive Metropolis–Hastings algorithm

In scenarios where setting the tuning parameters in the MH algorithm for a certain condition to be satisfied is challenging, we can do the tuning dynamically during the process of running the algorithm. For a Gaussian random-walk MH algorithm, the optimal scale matrix for the proposal distribution is $(2.38^2/d)\mathbf{\Sigma}^{[41]}$, where $\mathbf{\Sigma}$ is the covariance matrix of the parameters. Since $\mathbf{\Sigma}$ is not known in advance, Haario et al. (2001)^[24] proposed an adaptive Metropolis (AM) algorithm to approximate the optimal proposal. The proposal distribution used by Haario et al. (2001) has the form

$$Q_t(\boldsymbol{\theta}^* | \boldsymbol{\theta}) \sim N(\boldsymbol{\theta}, (2.38^2/d)\mathbf{\Sigma}_t + \epsilon \mathbf{I}_d),$$

where the constant $\epsilon > 0$ is some small value to avoid degeneracies, and in the numerical studies, we set the $\epsilon = 10^{-6}$. The $\mathbf{\Sigma}_t$ is the covariance matrix estimated using the previous t samples $\boldsymbol{\theta}^{(1)}, \dots, \boldsymbol{\theta}^{(t)}$, and the recursive formula for updating $\mathbf{\Sigma}_t$ is

$$\begin{aligned} \mathbf{\Sigma}_t &= \frac{1}{t-1} \sum_{i=1}^t (\boldsymbol{\theta}^{(i)} - \bar{\boldsymbol{\theta}}^{(t)})(\boldsymbol{\theta}^{(i)} - \bar{\boldsymbol{\theta}}^{(t)})^\top \\ &= \frac{t-2}{t-1} \mathbf{\Sigma}_{t-1} + \frac{1}{t} (\boldsymbol{\theta}^{(t)} - \bar{\boldsymbol{\theta}}^{(t-1)})(\boldsymbol{\theta}^{(t)} - \bar{\boldsymbol{\theta}}^{(t-1)})^\top, \end{aligned}$$

where $\bar{\boldsymbol{\theta}}^{(t)} = (t-1)/t \bar{\boldsymbol{\theta}}^{(t-1)} + \boldsymbol{\theta}^{(t)}/t$ is the arithmetic means of the first t samples of $\boldsymbol{\theta}$. Since the changes in scale matrix are negligible as $T \rightarrow \infty$, the ergodicity of the chain is preserved^[42]. In our numerical studies, we use this AM algorithm to tune the proposal of Gaussian random-walk MH adaptively.

3.1.3 Metropolis-adjusted Langevin algorithm

The Metropolis adjusted Langevin (MALA) algorithm, introduced by Roberts et al. (1998)^[43], constructs a MCMC algorithm by using a discretized Langevin diffusion process to propose new states. By incorporating the gradient information, the MALA algorithm quickly pushes the Markov chain to regions with high density and explores the parameter space more efficiently.

A Langevin diffusion with stationary distribution $p(\boldsymbol{\theta})$ is a continuous-time stochastic process satisfying the following stochastic differential equation^[3],

$$d\boldsymbol{\theta}^{(t)} = \frac{1}{2} \nabla_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta}^{(t)}) dt + d\mathbf{B}^{(t)},$$

where $\mathbf{B}^{(t)}$ represents a d -dimensional standard Brownian motion. In practice, it is necessary to consider discrete approximations of the Langevin diffusion. One common approach is the Euler method, which leads to the following discrete-time stochastic process,

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} + \frac{\epsilon^2}{2} \nabla_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta}^{(t)}) + \epsilon \mathbf{z}^{(t)}, \quad (3-1)$$

for a chosen discretization step $\epsilon > 0$, and $\mathbf{z}^{(t)} \stackrel{iid}{\sim} N(\mathbf{0}, \mathbf{I}_d)$. The discretized Langevin diffusion process may not guarantee to converge to $p(\boldsymbol{\theta})$. To address this issue, the MALA algorithm takes equation (3-1) as the proposal distribution in the MH algorithm and uses the accept / reject step to adjust the samples generated by the discretised process. Thus MALA is an MH algorithm, but not a Metropolis algorithm, since $\frac{Q(\boldsymbol{\theta}^{(t)}|\boldsymbol{\theta}^*)}{Q(\boldsymbol{\theta}^*|\boldsymbol{\theta}^{(t)})} \neq 1$.

To solve the problem that the components of $\boldsymbol{\theta}$ have different scales or highly correlated, a preconditioned version of MALA is proposed, that is

$$\boldsymbol{\theta}^* | \boldsymbol{\theta} \sim N_d \left(\boldsymbol{\theta} + \frac{\epsilon^2}{2} \boldsymbol{\Sigma} \nabla_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta}), \epsilon^2 \boldsymbol{\Sigma} \right),$$

where $\boldsymbol{\Sigma}$ is typically the estimate of the covariance matrix of $\boldsymbol{\theta}$. Let $\boldsymbol{\Sigma} = \mathbf{L}\mathbf{L}^\top$ be the Cholesky decomposition of $\boldsymbol{\Sigma}$. Denote $\tilde{\boldsymbol{\theta}} = \mathbf{L}^{-1}\boldsymbol{\theta}$ and then

$$\text{Var}(\tilde{\boldsymbol{\theta}}) = \mathbf{L}^{-1}(\mathbf{L}\mathbf{L}^\top)(\mathbf{L}^{-1})^\top = \mathbf{I}_d.$$

In other words, the preconditioned MALA is equivalent to the MALA algorithm with the orthogonal parametrization.

3.1.4 Hamiltonian Monte Carlo algorithm

HMC^[22] uses discretized Hamiltonian dynamics to propose new states. Specifically, HMC first embeds the target distribution $p(\boldsymbol{\theta})$ into Hamiltonian dynamics,

$$H(\boldsymbol{\theta}, \boldsymbol{\psi}) = U(\boldsymbol{\theta}) + K(\boldsymbol{\psi}). \quad (3-2)$$

Here H is the total energy of the system, $U(\boldsymbol{\theta}) = -\log p(\boldsymbol{\theta})$ is the potential energy with $\boldsymbol{\theta}$ called the position variables, and $K(\boldsymbol{\psi})$ is the kinetic energy with $\boldsymbol{\psi}$ the auxiliary momentum variable. A common choice for the distribution of $\boldsymbol{\psi}$, $p(\boldsymbol{\psi})$, is a multivariate Gaussian with mean zero and covariance matrix $\mathbf{M}$ ($\mathbf{M}$ is often called the “mass matrix”), and then $K(\boldsymbol{\psi}) = -\log p(\boldsymbol{\psi}) = \boldsymbol{\psi}^\top \mathbf{M}^{-1} \boldsymbol{\psi} / 2$. Then the HMC algorithm samples from the joint distribution of $\boldsymbol{\theta}$ and $\boldsymbol{\psi}$, i.e.,

$$p(\boldsymbol{\theta}, \boldsymbol{\psi}) = \exp \{-H(\boldsymbol{\theta}, \boldsymbol{\psi})\} = \exp \{-U(\boldsymbol{\theta})\} \exp \{-K(\boldsymbol{\psi})\} \propto p(\boldsymbol{\theta})p(\boldsymbol{\psi}),$$

where the marginal distribution of $\boldsymbol{\theta}$ is the target.

The Hamilton's equations are defined as

$$\frac{d\boldsymbol{\theta}}{dt} = \frac{\partial H}{\partial \boldsymbol{\psi}} = \mathbf{M}^{-1}\boldsymbol{\psi}, \quad (3-3)$$

$$\frac{d\boldsymbol{\psi}}{dt} = -\frac{\partial H}{\partial \boldsymbol{\theta}} = -\frac{\partial U}{\partial \boldsymbol{\theta}_i}. \quad (3-4)$$

Thus the gradient of $H(\boldsymbol{\theta}, \boldsymbol{\psi})$ with respect to $(\boldsymbol{\theta}, \boldsymbol{\psi})$ admits a physical interpretation as the time evolution of a Hamiltonian dynamic system with respect to fictitious time t . The leapfrog method is typically used to discretize the Hamilton's equations, and it proceeds as follows,

$$\boldsymbol{\psi}(t + \frac{\epsilon}{2}) = \boldsymbol{\psi}(t) + \frac{\epsilon}{2} \nabla_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta}(t)),$$

$$\boldsymbol{\theta}(t + \epsilon) = \boldsymbol{\theta}(t) + \epsilon \mathbf{M}^{-1} \boldsymbol{\psi}(t + \frac{\epsilon}{2}),$$

$$\boldsymbol{\psi}(t + \epsilon) = \boldsymbol{\psi}(t + \frac{\epsilon}{2}) + \frac{\epsilon}{2} \nabla_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta}(t + \epsilon)),$$

where ϵ is the step size.

Then each iteration of the corresponding HMC algorithm includes the following three steps.

- (1) Sample $\boldsymbol{\psi}$ from $p(\boldsymbol{\psi})$.
- (2) Give the current values of $(\boldsymbol{\theta}, \boldsymbol{\psi})$, propose new values $(\boldsymbol{\theta}^*, \boldsymbol{\psi}^*)$ using the Hamiltonian dynamics by applying the leapfrog method for L steps, and taking the output of the final step as the newly proposed values.
- (3) Accept $(\boldsymbol{\theta}^*, \boldsymbol{\psi}^*)$ with the probability

$$\min[1, \exp(-U(\boldsymbol{\theta}^*) + U(\boldsymbol{\theta}) - K(\boldsymbol{\psi}^*) + K(\boldsymbol{\psi}))].$$

Reversibility, conservation of total energy, and preservation of volume are fundamental properties of Hamiltonian dynamics. These properties guarantee that the HMC algorithm is reversible, transformation-invariant and with acceptance rate close to 1^[22]. As MALA, HMC also incorporates the gradient information of the target distribution. In addition, HMC moves multiple steps in the parameter space before accepting or rejecting, and thus favoring bigger jumps. HMC now becomes one of the most popular MCMC algorithms because of its high efficiency and favorable scalability for high-dimensional distributions^[44-46].

3.2 Estimation of Covariance Matrices

3.2.1 Estimate covariance matrix with burn-in sample

Sample covariance matrix obtained from the burn-in stage of the MCMC algorithm is the most straightforward estimate of the target covariance matrix, and the estimate can be accurate if the dimension of the target distribution is not too high. We typically run a chain of 10^6 plus a burn-in of 10^4 iterations in our numerical studies^[20]. The motivation of using burn-in is to make the chain approximately converge at the end of this phase. In fact, if the chain is far from convergence at the end of burn-in, the distribution of the subsequent samples may be more biased than expected.

3.2.2 Gaussian variational inference

Among the different VI approaches, Gaussian variational inference (GVI) is most widely used. The GVI tries to find the distribution that closest to the target posterior from a Gaussian distribution family. In this section, we will explore several variants of GVI and introduce the strength and applications of each.

3.2.2.1 GVI with Cholesky decomposed covariance

The idea behind GVI with Choleskey Decomposition Covariance (CGVI) is that we use the Choleskey decomposition for the covariance matrix Σ of the variational distribution $q_{\lambda}(\theta)$, expressed as $\Sigma = \mathbf{L}\mathbf{L}^T$, where $\mathbf{L}$ is a lower triangular matrix. To reduce the variance of the naive gradient estimator, here we use the reparameterization trick introduced in Section 2.2.3.2. Specifically, a sample $\theta \sim q_{\lambda}(\theta)$ can be parameterized as $\theta = g(\lambda, \epsilon) = \mu + \mathbf{L}\epsilon$, where $\epsilon \sim N_d(\mathbf{0}, \mathbf{I}_d)$, d is the dimension of θ and λ are the variational parameters including the mean μ and the non-zero elements of $\mathbf{L}$. We can obtain that the gradient of the evidence lower bound with respect to the parameter μ as follows,

$$\nabla_{\mu} LB(\lambda) = \mathbb{E}_{q_{\epsilon}}[\nabla_{\theta} h_{\lambda}(\theta)], \text{ with } \theta = \mu + \mathbf{L}\epsilon. \quad (3-5)$$

For the gradient with respect to the parameter $\mathbf{L}$, we introduce some preliminary notations in the matrix computations. If $\mathbf{C}$ is a matrix of dimension $d \times d$, then the vector $\text{vech}(\mathbf{C})$ consists of $\frac{1}{2}d(d+1)$ elements obtained by stacking the columns of the lower triangular part of $\mathbf{C}$. On the other hand, the vector $\text{vec}(\mathbf{C})$ is obtained by stacking the columns of matrix $\mathbf{C}$ from left to right. In addition, $\mathbf{C} \otimes \mathbf{D}$ denotes the Kronecker product of matrix $\mathbf{C}$ and $\mathbf{D}$ ^[12]. Furthermore, for any matrices $\mathbf{C}$, $\mathbf{D}$ and $\mathbf{X}$ of suitable sizes, we have the

property that $\text{vec}(\mathbf{C}\mathbf{X}\mathbf{D}) = (\mathbf{D}^\top \otimes \mathbf{C}) \text{vec}(\mathbf{X})^{[12]}$. Using these notations, we get

$$\mathbf{L}\boldsymbol{\epsilon} = \text{vec}(\mathbf{I}_d \mathbf{L}\boldsymbol{\epsilon}) = (\boldsymbol{\epsilon}^\top \otimes \mathbf{I}_d) \text{vec}(\mathbf{L}).$$

Consequently, we have

$$\nabla_{\text{vec}(\mathbf{L})} g(\boldsymbol{\lambda}, \boldsymbol{\epsilon}) = \boldsymbol{\epsilon}^\top \otimes \mathbf{I}_d,$$

leading to

$$\begin{aligned} \nabla_{\text{vec}(\mathbf{L})} \text{LB}(\boldsymbol{\lambda}) &= \mathbb{E}_{q_\epsilon} [\nabla_{\text{vec}(\mathbf{L})} g(\boldsymbol{\lambda}, \boldsymbol{\epsilon})^\top \nabla_{\boldsymbol{\theta}} h_{\boldsymbol{\lambda}}(\boldsymbol{\theta})] \\ &= \mathbb{E}_{q_\epsilon} [(\boldsymbol{\epsilon} \otimes \mathbf{I}_d) \nabla_{\boldsymbol{\theta}} h_{\boldsymbol{\lambda}}(\boldsymbol{\theta})] \\ &= \mathbb{E}_{q_\epsilon} [\text{vec}(\nabla_{\boldsymbol{\theta}} h_{\boldsymbol{\lambda}}(\boldsymbol{\theta}) \boldsymbol{\epsilon}^\top)], \text{ with } \boldsymbol{\theta} = \boldsymbol{\mu} + \mathbf{L}\boldsymbol{\epsilon}. \end{aligned}$$

This implies that

$$\nabla_{\text{vech}(\mathbf{L})} \text{LB}(\boldsymbol{\lambda}) = \mathbb{E}_{q_\epsilon} [\text{vech}(\nabla_{\boldsymbol{\theta}} h_{\boldsymbol{\lambda}}(\boldsymbol{\theta}) \boldsymbol{\epsilon}^\top)]. \quad (3-6)$$

In conclusion, we arrive at the Cholesky Gaussian variational inference algorithm, referred to as CGVI^[12].

Algorithm 3.1 Cholesky Gaussian variational inference^[12]

- 1: Initialize $\boldsymbol{\lambda}^{(0)} = (\boldsymbol{\mu}^{(0)}, \text{vech}(\mathbf{L}^{(0)}))$.
 - 2: **while** stopping rule is not satisfied **do**:
 - 3: Generate $\boldsymbol{\epsilon}_s \sim N(\mathbf{0}, \mathbf{I})$, for $s = 1, 2, \dots, S$.
 - 4: Construct unbiased estimates for the gradient in equations (3-5) and (3-6) as $\nabla_{\boldsymbol{\mu}} \widehat{\text{LB}}(\boldsymbol{\lambda}^{(t)}) = \frac{1}{S} \sum_{s=1}^S \nabla_{\boldsymbol{\theta}} h_{\boldsymbol{\lambda}}(\boldsymbol{\theta}_s)$, $\nabla_{\text{vech}(\mathbf{L})} \widehat{\text{LB}}(\boldsymbol{\lambda}^{(t)}) = \frac{1}{S} \sum_{s=1}^S \text{vech}(\nabla_{\boldsymbol{\theta}} h_{\boldsymbol{\lambda}}(\boldsymbol{\theta}_s) \boldsymbol{\epsilon}_s^\top)$, where $\boldsymbol{\theta}_s = \boldsymbol{\mu}^{(t)} + \mathbf{L}^{(t)} \boldsymbol{\epsilon}_s$ using samples of $\{\boldsymbol{\epsilon}_s, s = 1, \dots, S\}$.
 - 5: Set $\boldsymbol{\rho}^{(t)} = (\boldsymbol{\rho}_{\boldsymbol{\mu}}^{(t)}, \boldsymbol{\rho}_{\text{vech}(\mathbf{L})}^{(t)})$.
 - 6: Update $\boldsymbol{\mu}^{(t+1)} = \boldsymbol{\mu}^{(t)} + \boldsymbol{\rho}_{\boldsymbol{\mu}}^{(t)} \odot \nabla_{\boldsymbol{\mu}} \widehat{\text{LB}}(\boldsymbol{\lambda}^{(t)})$.
 - 7: Update $\text{vech}(\mathbf{L})^{(t+1)} = \text{vech}(\mathbf{L})^{(t)} + \boldsymbol{\rho}_{\text{vech}(\mathbf{L})}^{(t)} \odot \nabla_{\text{vech}(\mathbf{L})} \widehat{\text{LB}}(\boldsymbol{\lambda}^{(t)})$.
 - 8: Set $\boldsymbol{\lambda}^{(t+1)} = (\boldsymbol{\mu}^{(t+1)}, \text{vech}(\mathbf{L})^{(t+1)})$, $t \rightarrow t + 1$.
 - 9: **end while**
-

In our numerical studies, we set the learning rates $\boldsymbol{\rho}_t$ by a simple adaptive-learning method introduced in 2.2.3.3 and set $\beta_1 = 0.9$, $\beta_2 = 0.9$, $\epsilon_0 = 0.01$ and $\tau = 1000$, $S = 100$. As described in section 2.2.3.4, we caculate the moving averages of LB , $\overline{\text{LB}}$, keep a record of the maximum $\overline{\text{LB}}$ so far, denoted by $\overline{\text{LB}}_{\max}$ and stop the algorithm if $\overline{\text{LB}}$ falls below $\overline{\text{LB}}$ for P consecutive times. In addition, we set the window size of the moving average to be 50 and $P = 50$.

3.2.2.2 GVI with factor decomposed covariance

In the case of CGVI mentioned in section 3.2.2.1, there are $d + d \times (d + 1) / 2$ variational parameters. When the dimension d of the variational distribution is large, the computational complexity of large matrices and the increased memory requirements will limit the use of the CGVI method.

Here we introduce an alternative method, GVI with Factor Decomposed Covariance (FGVI) which was first proposed by Ong et al. (2018)^[13]. The idea behind FGVI is the factor decomposition, which is expressed as

$$\Sigma = \mathbf{B}\mathbf{B}^\top + \mathbf{C}^2. \quad (3-7)$$

Here, $\mathbf{B}$ represents a factor loading matrix of dimensions $d \times f$, where f denotes the number of factors, $f \ll d$. Additionally, $\mathbf{C}$ is a diagonal matrix, that is, $\mathbf{C} = \text{diag}(c_1, \dots, c_d)$. The number of variational parameters reduces from $d + d \times (d + 1) / 2$ for CGVI to $(f + 2)d$ for FGVI.

With factor $f = 1$ in FGVI, Tran et al. (2020)^[47] show that it is straightforward to derive the natural gradient and the corresponding GVI algorithm. Now the factor decomposition of Σ becomes

$$\Sigma = \mathbf{b}\mathbf{b}^\top + \mathbf{C}^2, \quad \mathbf{C} = \text{diag}(\mathbf{c}),$$

where $\mathbf{b} = (b_1, \dots, b_d)^\top$ and $\mathbf{c} = (c_1, \dots, c_d)^\top$, and the variational parameter is $\lambda = (\mu^\top, \mathbf{b}^\top, \mathbf{c}^\top)$. Similarly, using the reparameterization trick, $\theta \sim N(\mu, \Sigma)$ can be calculated as follows,

$$\theta = g(\lambda, \epsilon) = \mu + \epsilon_1 \mathbf{b} + \mathbf{c} \odot \epsilon_2,$$

where $\epsilon = (\epsilon_1, \epsilon_2^\top)^\top \sim N_{d+1}(\mathbf{0}, \mathbf{I})$. Recall that the reparameterization gradient derived in section 2.2.3.2 is

$$\nabla_\lambda \text{LB}(\lambda) = \mathbb{E}_{\epsilon \sim q_\epsilon} (\nabla_\lambda g(\lambda, \epsilon)^\top \nabla_\theta h_\lambda(\theta)).$$

Since

$$\nabla_\mu g(\lambda, \epsilon) = \mathbf{I}_d, \quad \nabla_{\mathbf{b}} g(\lambda, \epsilon) = \epsilon_1 \mathbf{I}_d, \quad \nabla_{\mathbf{c}} g(\lambda, \epsilon) = \text{diag}(\epsilon_2),$$

we have the reparameterization gradient for $f = 1$ as follows,

$$\nabla_\lambda \text{LB}(\lambda) = \mathbb{E}_{q_\epsilon} \begin{pmatrix} \nabla_\theta h_\lambda(\mu + \epsilon_1 \mathbf{b} + \mathbf{c} \odot \epsilon_2) \\ \epsilon_1 \nabla_\theta h_\lambda(\mu + \epsilon_1 \mathbf{b} + \mathbf{c} \odot \epsilon_2) \\ \epsilon_2 \odot \nabla_\theta h_\lambda(\mu + \epsilon_1 \mathbf{b} + \mathbf{c} \odot \epsilon_2) \end{pmatrix}.$$

The gradient of function $h_\lambda(\boldsymbol{\theta})$ is

$$\nabla_{\boldsymbol{\theta}} h_\lambda(\boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}} h(\boldsymbol{\theta}) - \nabla_{\boldsymbol{\theta}} \log q_\lambda(\boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}} \log(p(\boldsymbol{\theta})p(y | \boldsymbol{\theta})) - \nabla_{\boldsymbol{\theta}} \log q_\lambda(\boldsymbol{\theta}),$$

where the first term depends on the target distribution, and the second term is $\nabla_{\boldsymbol{\theta}} \log q_\lambda(\boldsymbol{\theta}) = -\boldsymbol{\Sigma}^{-1}(\boldsymbol{\theta} - \boldsymbol{\mu})$. We can avoid directly computing the inverse $\boldsymbol{\Sigma}^{-1}$ based on the Woodbury formula “ $\boldsymbol{\Sigma}^{-1} = \mathbf{C}^{-2} - \frac{1}{1+\mathbf{b}^\top \mathbf{C}^{-2} \mathbf{b}} \mathbf{C}^{-2} \mathbf{b} \mathbf{b}^\top \mathbf{C}^{-2}$ ”. Then we get

$$\nabla_{\boldsymbol{\theta}} \log q_\lambda(\boldsymbol{\theta}) = -(\boldsymbol{\theta} - \boldsymbol{\mu}) \odot \mathbf{c}^{-2} + \frac{(\mathbf{b} \odot \mathbf{c}^{-2})^\top (\boldsymbol{\theta} - \boldsymbol{\mu})}{1 + (\mathbf{b} \odot \mathbf{c}^{-1})^\top (\mathbf{b} \odot \mathbf{c}^{-1})} (\mathbf{b} \odot \mathbf{c}^{-2}),$$

with $\mathbf{c}^{-1} := (1/c_1, \dots, 1/c_d)^\top$ and $\mathbf{c}^{-2} := (1/c_1^2, \dots, 1/c_d^2)^\top$.

In some cases, the FGV algorithm with $f = 1$ may fail to capture the complex structure of the target distribution. Therefore, it is necessary to increase the number of factors in FGV. When $f \neq 1$, to use reparametrization trick, we draw $\boldsymbol{\theta} \sim N(\boldsymbol{\mu}, \mathbf{B}\mathbf{B}^\top + \mathbf{C}^2)$ by first drawing $\boldsymbol{\zeta} = (\mathbf{z}^\top, \boldsymbol{\epsilon}^\top)^\top \sim q(\boldsymbol{\zeta})$, where $\boldsymbol{\zeta} \sim N(\mathbf{0}, \mathbf{I})$, and then we calculate $\boldsymbol{\theta} = g(\boldsymbol{\zeta}, \boldsymbol{\lambda}) = \boldsymbol{\mu} + \mathbf{B}\mathbf{z} + \mathbf{c} \odot \boldsymbol{\epsilon}$. Thus the gradient of $g(\boldsymbol{\zeta}, \boldsymbol{\lambda})$ becomes

$$\nabla_{\boldsymbol{\mu}} g(\boldsymbol{\zeta}, \boldsymbol{\lambda}) = \mathbf{I},$$

$$\nabla_{\text{vec}(\mathbf{B})} g(\boldsymbol{\zeta}, \boldsymbol{\lambda}) = \mathbf{z}^\top \otimes \mathbf{I}.$$

In addition, in the case of FGV, $\nabla_{\boldsymbol{\theta}} \log q_\lambda(\boldsymbol{\theta}) = -(\mathbf{B}\mathbf{B}^\top + \mathbf{C}^2)^{-1}(\boldsymbol{\theta} - \boldsymbol{\mu})$, we can obtain gradients of the evidence lower bound with respect to $\boldsymbol{\mu}$, $\mathbf{B}$ and $\mathbf{c}$ as

$$\begin{aligned} \nabla_{\boldsymbol{\mu}} \text{LB}(\boldsymbol{\lambda}) &= \mathbb{E}_{q_{\boldsymbol{\zeta}}} [\nabla_{\boldsymbol{\theta}} \log h(\boldsymbol{\mu} + \mathbf{B}\mathbf{z} + \mathbf{c} \odot \boldsymbol{\epsilon}) \\ &\quad + (\mathbf{B}\mathbf{B}^\top + \mathbf{C}^2)^{-1}(\mathbf{B}\mathbf{z} + \mathbf{c} \odot \boldsymbol{\epsilon})], \end{aligned} \quad (3-8)$$

$$\begin{aligned} \nabla_{\mathbf{B}} \text{LB}(\boldsymbol{\lambda}) &= \mathbb{E}_{q_{\boldsymbol{\zeta}}} [\nabla_{\boldsymbol{\theta}} \log h(\boldsymbol{\mu} + \mathbf{B}\mathbf{z} + \mathbf{c} \odot \boldsymbol{\epsilon}) \mathbf{z}^\top \\ &\quad + (\mathbf{B}\mathbf{B}^\top + \mathbf{C}^2)^{-1}(\mathbf{B}\mathbf{z} + \mathbf{c} \odot \boldsymbol{\epsilon}) \mathbf{z}^\top], \end{aligned} \quad (3-9)$$

$$\begin{aligned} \nabla_{\mathbf{c}} \text{LB}(\boldsymbol{\lambda}) &= \mathbb{E}_{q_{\boldsymbol{\zeta}}} [\text{Diag}(\nabla_{\boldsymbol{\theta}} \log h(\boldsymbol{\mu} + \mathbf{B}\mathbf{z} + \mathbf{c} \odot \boldsymbol{\epsilon}) \boldsymbol{\epsilon}^\top \\ &\quad + (\mathbf{B}\mathbf{B}^\top + \mathbf{C}^2)^{-1}(\mathbf{B}\mathbf{z} + \mathbf{c} \odot \boldsymbol{\epsilon}) \boldsymbol{\epsilon}^\top)]. \end{aligned} \quad (3-10)$$

Here $\text{Diag}(\cdot)$ represents the vector of diagonal elements in a square matrix and $(\mathbf{B}\mathbf{B}^\top + \mathbf{C}^2)^{-1}$ can be efficiently computed using the Woodbury formula,

$$(\mathbf{B}\mathbf{B}^\top + \mathbf{C}^2)^{-1} = \mathbf{C}^{-2} - \mathbf{C}^{-2} \mathbf{B} (\mathbf{I} + \mathbf{B}^\top \mathbf{C}^{-2} \mathbf{B})^{-1} \mathbf{B}^\top \mathbf{C}^{-2}.$$

Then we can use the SGA algorithm with specific adaptive learning rate method to optimize the evidence lower bound, leading to the following algorithm, referred to as FGV.

Algorithm 3.2 GVI with factor decomposed covariance structure^[13]

- 1: Initialize $\lambda^{(0)} = (\mu^{(0)}, \mathbf{B}^{(0)}, \mathbf{c}^{(0)})$.
 - 2: **while** stopping rule is not satisfied **do**
 - 3: Generate $\zeta_s \sim N(\mathbf{0}, \mathbf{I})$, for $s = 1, 2, \dots, S$.
 - 4: Construct unbiased gradient estimates of gradients in equations (3-9), (3-10) and (3-11) using samples $\{\zeta_s, s = 1, \dots, S\}$.
 - 5: Set adaptive learning rate $\rho^{(t)} = (\rho_\mu^{(t)}, \rho_B^{(t)}, \rho_c^{(t)})$ using ADADELTA method which is introduced in section 2.2.3.3 or other methods.
 - 6: Update $\mu^{(t+1)} = \mu^{(t)} + \rho_\mu^{(t)} \odot \nabla_\mu \text{LB}(\lambda^{(t)})$.
 - 7: Update $\mathbf{B}^{(t+1)} = \mathbf{B}^{(t)} + \rho_B^{(t)} \odot \nabla_B \widehat{\text{LB}}(\lambda^{(t)})$ for elements of $\mathbf{B}^{(t+1)}$ on or below the diagonal, with the upper triangle of $\mathbf{B}^{(t+1)}$ fixed at zero.
 - 8: Update $\mathbf{c}^{(t+1)} = \mathbf{c}^{(t)} + \rho_c^{(t)} \odot \nabla_c \widehat{\text{LB}}(\lambda^{(t)})$.
 - 9: Set $\lambda^{(t+1)} = (\mu^{(t+1)}, \mathbf{B}^{(t+1)}, \mathbf{c}^{(t+1)})$, $t \rightarrow t + 1$.
 - 10: **end while**
-

In our numerical studies, when the $f = 1$, we set the learning rates ρ_t by a simple adaptive-learning method introduced in 2.2.3.3 and set $\beta_1 = 0.9$, $\beta_2 = 0.9$, $\epsilon_0 = 0.05$ and $\tau = 100$, $S = 100$ and when the $f \neq 1$, we set the learning rate by ADADELTA method which is introduced in section 2.2.3.3, and we set $v = 0.95$, $\delta = 10^{-6}$, $S = 50$. As described in section 2.2.3.4, we calculate the moving averages of LB , $\overline{\text{LB}}$, keep a record of the maximum $\overline{\text{LB}}$ so far, denoted by $\overline{\text{LB}}_{\max}$ and stop the algorithm if $\overline{\text{LB}}$ falls below $\overline{\text{LB}}$ for P consecutive times. In addition, we set the window size of the moving average to be 50 and $P = 50$.

3.2.2.3 GVI with sparse precision matrices

GVI with sparse precision matrices was proposed by Tan et al. (2018)^[14]. Assume that the variational approximation distribution $q(\cdot)$ of the target is a multivariate Gaussian distribution $N(\mu, \Omega^{-1})$, where Ω denotes the precision matrix. Considering the decomposition of Ω as $\Omega = \mathbf{L}\mathbf{L}^\top$, where $\mathbf{L}$ is the Cholesky factor, Tan et al. (2018)^[14] imposes sparsity on $\mathbf{L}$ to achieve conditional independence in Ω . Then $q(\theta \mid \mu, \mathbf{L}) = |\mathbf{L}|f(\mathbf{L}^\top(\theta - \mu))$, with f the joint density function of a d -dimensional vector of independent standard Gaussian variables. Then we can obtain the ELBO as follows^[14],

$$\begin{aligned} \text{LB}(\mu, \mathbf{L}) &= \mathbb{E}_f [\log h(\mu + \mathbf{L}^{-\top} \epsilon) - \log q(\mu + \mathbf{L}^{-\top} \epsilon \mid \mu, \mathbf{L})] \\ &= \mathbb{E}_f [\log h(\mu + \mathbf{L}^{-\top} \epsilon)] - \log |\mathbf{L}| + C, \end{aligned}$$

where $\boldsymbol{\epsilon} = \mathbf{L}^\top(\boldsymbol{\theta} - \boldsymbol{\mu}) \sim f$ and C is a constant not depending on the variational parameters. Since

$$\nabla_{\boldsymbol{\theta}} \log q(\boldsymbol{\theta}) = -\mathbf{L}\mathbf{L}^\top(\boldsymbol{\theta} - \boldsymbol{\mu}) = -\mathbf{L}\boldsymbol{\epsilon},$$

the gradients^[14] of $\nabla_{\boldsymbol{\mu}}\text{LB}$ and $\nabla_{\mathbf{L}}\text{LB}$ are

$$\begin{aligned} \nabla_{\boldsymbol{\mu}}\text{LB} &= \nabla_{\boldsymbol{\mu}}\mathbb{E}_f [\log h(\boldsymbol{\mu} + \mathbf{L}^{-\top}\boldsymbol{\epsilon}) - \log q(\boldsymbol{\mu} + \mathbf{L}^{-\top}\boldsymbol{\epsilon})] \\ &= \mathbb{E}_f [\nabla_{\boldsymbol{\theta}} \log h(\boldsymbol{\mu} + \mathbf{L}^{-\top}\boldsymbol{\epsilon}) - \nabla_{\boldsymbol{\theta}} \log q(\boldsymbol{\mu} + \mathbf{L}^{-\top}\boldsymbol{\epsilon})] \\ &= \mathbb{E}_f [\nabla_{\boldsymbol{\theta}} \log h(\boldsymbol{\mu} + \mathbf{L}^{-\top}\boldsymbol{\epsilon}) + \mathbf{L}\boldsymbol{\epsilon}], \end{aligned}$$

$$\begin{aligned} \nabla_{\mathbf{L}}\text{LB} &= \nabla_{\mathbf{L}}\mathbb{E}_f [\log h(\boldsymbol{\mu} + \mathbf{L}^{-\top}\boldsymbol{\epsilon}) - \log q(\boldsymbol{\mu} + \mathbf{L}^{-\top}\boldsymbol{\epsilon})] \\ &= -\mathbb{E}_f \left\{ \mathbf{L}^{-\top}\boldsymbol{\epsilon} [\nabla_{\boldsymbol{\theta}} \log h(\boldsymbol{\mu} + \mathbf{L}^{-\top}\boldsymbol{\epsilon}) - \nabla_{\boldsymbol{\theta}} \log q(\boldsymbol{\mu} + \mathbf{L}^{-\top}\boldsymbol{\epsilon})]^\top \mathbf{L}^{-\top} \right\} \\ &= -\mathbb{E}_f \left\{ \mathbf{L}^{-\top}\boldsymbol{\epsilon} [\nabla_{\boldsymbol{\theta}} \log h(\boldsymbol{\mu} + \mathbf{L}^{-\top}\boldsymbol{\epsilon}) + \mathbf{L}\boldsymbol{\epsilon}]^\top \mathbf{L}^{-\top} \right\}. \end{aligned}$$

Thus we can obtain the unbiased estimators of $\nabla_{\boldsymbol{\mu}}L$ and $\nabla_{\mathbf{L}}L$ as follows,

$$\hat{g}_{\boldsymbol{\mu}} = \nabla_{\boldsymbol{\theta}} \log h(\boldsymbol{\mu} + \mathbf{L}^{-\top}\boldsymbol{\epsilon}) + \mathbf{L}\boldsymbol{\epsilon},$$

$$\hat{g}_{\mathbf{L}} = -\mathbf{L}^{-\top}\boldsymbol{\epsilon} [\nabla_{\boldsymbol{\theta}} \log h(\boldsymbol{\mu} + \mathbf{L}^{-\top}\boldsymbol{\epsilon}) + \mathbf{L}\boldsymbol{\epsilon}]^\top \mathbf{L}^{-\top}.$$

Finally, we can obtain the algorithm of GVI with sparse precision matrices as follows^[14].

Algorithm 3.3 GVI with sparse precision matrices^[14]

- 1: Initialize $\boldsymbol{\mu}^{(0)} = \mathbf{0}$, $\mathbf{L}^{(0)} = \mathbf{I}_d$ and $\mathbf{L}'^{(0)} = \mathbf{0}$.
 - 2: **While** stopping rule is not satisfied **do**:
 - 3: Generate $\boldsymbol{\epsilon} \sim N(\mathbf{0}, \mathbf{I}_d)$.
 - 4: Calculate $\boldsymbol{\theta}^{(t)} = \boldsymbol{\mu}^{(t-1)} + \mathbf{L}^{(t-1)-\top}\boldsymbol{\epsilon}$.
 - 5: Calculate $\mathbf{g}_{\boldsymbol{\mu}}^{(t)} = \nabla_{\boldsymbol{\theta}} \log h(\boldsymbol{\theta}^{(t)}) + \mathbf{L}^{(t-1)}\boldsymbol{\epsilon}$.
 - 6: Update $\boldsymbol{\mu}^{(t)} = \boldsymbol{\mu}^{(t-1)} + \boldsymbol{\rho}_t \mathbf{g}_{\boldsymbol{\mu}}^{(t)}$.
 - 7: Set $\mathbf{g}_{\mathbf{L}'}^{(t)} = -\mathbf{L}^{(t-1)-\top}\boldsymbol{\epsilon}(\mathbf{L}^{(t-1)-1}\mathbf{g}_{\boldsymbol{\mu}}^{(t)})^\top$.
 - 8: Set $\text{diag}(\mathbf{g}_{\mathbf{L}'}^{(t)}) = \text{diag}(\mathbf{g}_{\mathbf{L}'}^{(t)}) \odot \text{diag}(\mathbf{L}^{(t-1)})$.
 - 9: Update $\mathbf{L}'^{(t)} = \mathbf{L}'^{(t-1)} + \boldsymbol{\rho}_t \mathbf{g}_{\mathbf{L}'}^{(t)}$.
 - 10: Update $\mathbf{L}^{(t)} = \mathbf{L}'^{(t)}$.
 - 11: Update $\text{diag}(\mathbf{L}^{(t)}) = \exp(\text{diag}(\mathbf{L}'^{(t)}))$.
 - 12: $t \rightarrow t + 1$.
 - 13: **end while**
-

In practice, we set the learning rates $\boldsymbol{\rho}_t$ by the ADADELTA method introduced in section 2.2.3.3. As described in section 2.2.3.4, we calculate the moving averages of LB, $\overline{\text{LB}}$, keep a record of the maximum $\overline{\text{LB}}$ so far, denoted by $\overline{\text{LB}}_{\max}$ and stop the algorithm

if $\overline{LB}$ falls below $\overline{LB}$ for P consecutive times. In our numerical study, we set the window size of the moving average to be 1000 and $P=50$.

3.3 Linear Transformations in HMC via Gaussian Variational Inference

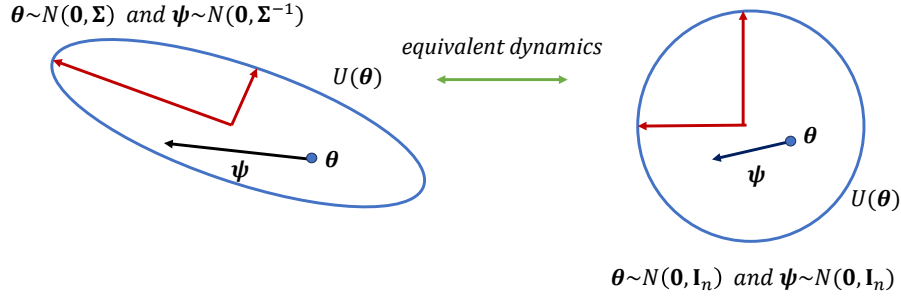


Figure 3-1 Hamiltonian dynamic with position variables $\theta \sim N(0, \Sigma)$ and momentum variables $\psi \sim N(0, \Sigma^{-1})$ is equivalent to the ideal HMC dynamic where both θ and ψ have isotropic covariance. This equivalence demonstrates that the ideal momentum covariance for HMC is the inverse of the covariance of position parameter θ ^[48].

Hamiltonian dynamics are equivalent under linear transformation. Specifically, if the position variables θ are multiplied by some nonsingular matrix $\mathbf{A}$, provided that simultaneously the corresponding momentum variables ψ being transformed by $(\mathbf{A}^\top)^{-1}$, then the new Hamiltonian dynamics will behave equivalently to the original dynamics. The phenomenon is illustrated in Figure 3-1.

This property can be demonstrated as follows. Consider the linear transformation $\theta' = \mathbf{A}\theta$. The probability density for θ' , denoted by $p'(\theta')$, can be derived as $p'(\theta') = p(\mathbf{A}^{-1}\theta') / |\det(\mathbf{A})|$, where $p(\theta) = \exp\{-U(\theta)\}$, so that $U'(\theta') = U(\mathbf{A}^{-1}\theta')$. The term $\log |\det(\mathbf{A})|$ is omitted from the potential energy because it is constant. For the distribution of the momentum variable ψ , if we adopt the setup described in section 3.1.4, i.e., a zero-mean multivariate Gaussian with covariance matrix $\mathbf{M}$, then the kinetic energy becomes $K(\psi) = \psi^\top \mathbf{M}^{-1} \psi / 2$. We transform ψ using $(\mathbf{A}^\top)^{-1}$, i.e., $\psi' = (\mathbf{A}^\top)^{-1} \psi$, and then $K'(\psi') = K(\mathbf{A}^\top \psi')$ is the new kinetic energy and

$$K'(\psi') = (\mathbf{A}^\top \psi')^\top \mathbf{M}^{-1} (\mathbf{A}^\top \psi') / 2 = (\psi')^\top (\mathbf{A} \mathbf{M}^{-1} \mathbf{A}^\top) \psi' / 2 = (\psi')^\top (\mathbf{M}')^{-1} \psi' / 2, \quad (3-11)$$

where $\mathbf{M}' = (\mathbf{A} \mathbf{M}^{-1} \mathbf{A}^\top)^{-1} = (\mathbf{A}^{-1})^\top \mathbf{M} \mathbf{A}^{-1}$. Then the Hamiltonian equations of (θ, ψ) by following the Hamiltonian dynamics of (θ', ψ') become

$$\frac{d\theta}{dt} = \mathbf{A}^{-1} \frac{d\theta'}{dt} = \mathbf{A}^{-1} (\mathbf{M}')^{-1} \psi' = \mathbf{A}^{-1} (\mathbf{A} \mathbf{M}^{-1} \mathbf{A}^\top) (\mathbf{A}^\top)^{-1} \psi = \mathbf{M}^{-1} \psi,$$

$$\frac{d\boldsymbol{\psi}}{dt} = \mathbf{A}^\top \frac{d\boldsymbol{\psi}'}{dt} = -\mathbf{A}^\top \nabla U'(\boldsymbol{\theta}') = -\mathbf{A}^\top (\mathbf{A}^{-1})^\top \nabla U(\mathbf{A}^{-1}\boldsymbol{\theta}') = -\nabla U(\boldsymbol{\theta}),$$

which are same as the Hamiltonian dynamics for $(\boldsymbol{\theta}, \boldsymbol{\psi})$ in equations (3-3) and (3-4). This property can provide us suggestions for implementing HMC if there is prior information available regarding the scales and correlations of the variables in the target distribution^[22]. Therefore, we utilize this property in our new HMC algorithm.

Firstly, we use the GVI to derive a Gaussian variational approximation distribution $q(\boldsymbol{\theta}) = N(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ of the target distribution $p(\boldsymbol{\theta})$. Then we use the $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ in $q(\boldsymbol{\theta})$ to facilitate the design of the HMC algorithm for sampling $p(\boldsymbol{\theta})$. We use $\boldsymbol{\mu}$ as the initial state of $\boldsymbol{\theta}$ in the algorithm. There are two equivalent approaches to incorporating $\boldsymbol{\Sigma}$ in the HMC. The first approach is to find the Cholesky decomposition, $\boldsymbol{\Sigma} = \mathbf{L}\mathbf{L}^\top$, where $\mathbf{L}$ is lower-triangular Cholesky factor, and let $\boldsymbol{\theta}' = \mathbf{L}^{-1}\boldsymbol{\theta}$. The new position variables tend to be nearly uncorrelated with variance close to one provided that $\boldsymbol{\Sigma}$ is an accurate estimate of the covariance matrix of $\boldsymbol{\theta}$. Thus the potential energy becomes $U'(\boldsymbol{\theta}') = -\log p'(\boldsymbol{\theta}') = -\log p(\mathbf{L}^{-1}\boldsymbol{\theta}) = U(\mathbf{L}^{-1}\boldsymbol{\theta})$ and then define our kinetic energy to be $K(\boldsymbol{\psi}) = \boldsymbol{\psi}^\top \boldsymbol{\psi} / 2$, that is, the momentum variables are independent. The second approach is to keep the original $\boldsymbol{\theta}$ variables so that the potential energy is $U(\boldsymbol{\theta}) = -\log p(\boldsymbol{\theta})$, but define the kinetic energy as $K(\boldsymbol{\psi}) = \boldsymbol{\psi}^\top \boldsymbol{\Sigma} \boldsymbol{\psi} / 2$, i.e, the distribution of the momentum variables is Gaussian with mean $\mathbf{0}$ and covariance $\boldsymbol{\Sigma}^{-1}$. We use the second approach in our numerical studies. Even when the $\boldsymbol{\Sigma}$ is not an accurate estimate of the covariance of $\boldsymbol{\theta}$, employing this strategy could still improve the performance of HMC, see Neal et al (2010)^[22]. Then our new HMC algorithm proceeds as Algorithm 3.4 below.

In the numerical studies, we compare the efficiency of the HMC algorithms with the covariance matrices $\boldsymbol{\Sigma}$ used for linear transformation estimated from MCMC burn-in samples, and estimated using GVI respectively. In addition to deciding the covariance matrix $\mathbf{M}$ for the momentum variable $\boldsymbol{\psi}$, another challenge in implementing HMC lies in selecting suitable values for ϵ and L . For tuning HMC, it is common to perform preliminary runs with trial values. Ideally, we choose such values of ϵ and L that the state after one complete HMC iteration is nearly independent of the previous state^[22], i.e, to tune ϵ and L so that ρ_1 of $\boldsymbol{\theta}$ is approximately equal to 0.

Algorithm 3.4 Linear transformation in HMC via GVI

```

1: Using the GVI to obtain a variational approximation  $N(\boldsymbol{\mu}, \boldsymbol{\Sigma})$  of  $p(\boldsymbol{\theta})$ 
2: Initialize  $\boldsymbol{\theta}$  with  $\boldsymbol{\mu}$ 
3:  $U(\boldsymbol{\theta}) = -\log p(\boldsymbol{\theta})$ 
4: for  $t = 1$  to  $T$  do
5:    $\boldsymbol{\psi} \sim N(\mathbf{0}, \boldsymbol{\Sigma}^{-1})$  ▷ momentum variables
6:    $K(\boldsymbol{\psi}) = \boldsymbol{\psi}^\top \boldsymbol{\Sigma} \boldsymbol{\psi} / 2$  ▷ Kinetic energy at the beginning of the trajectory
7:    $\boldsymbol{\theta}^* \leftarrow \boldsymbol{\theta}$ 
8:    $\boldsymbol{\psi}^* \leftarrow \boldsymbol{\psi} + \epsilon \nabla_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta}^*) / 2$  ▷ Half step for momentum variable
9:   for  $l = 1$  to  $L$  do
10:     $\boldsymbol{\theta}^* \leftarrow \boldsymbol{\theta}^* + \epsilon \boldsymbol{\Sigma} \boldsymbol{\psi}^*$  ▷ Full step for position variable
11:    if  $l \neq L$  then
12:       $\boldsymbol{\psi}^* \leftarrow \boldsymbol{\psi}^* + \epsilon \nabla_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta}^*)$ 
13:    end if
14:  end for
15:   $\boldsymbol{\psi}^* \leftarrow \boldsymbol{\psi}^* + \epsilon \nabla_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta}^*) / 2$  ▷ Half step for momentum at the end
16:   $\boldsymbol{\psi}^* \leftarrow -\boldsymbol{\psi}^*$  ▷ Negate momentum to make the proposal symmetric
17:   $U(\boldsymbol{\theta}^*) = -\log p(\boldsymbol{\theta}^*)$ 
18:   $K(\boldsymbol{\psi}^*) = \boldsymbol{\psi}^{*\top} \boldsymbol{\Sigma} \boldsymbol{\psi}^* / 2$ 
19:  if  $\text{runif}(1) < \exp(U(\boldsymbol{\theta}) + K(\boldsymbol{\psi}) - U(\boldsymbol{\theta}^*) - K(\boldsymbol{\psi}^*))$  then
20:     $U(\boldsymbol{\theta}) \leftarrow U(\boldsymbol{\theta}^*)$ 
21:     $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta}^*$  ▷ Accept the value
22:  end if
23: end for
    
```

3.4 Efficient Bayesian Sampling Using VAE to Assist MCMC Methods

3.4.1 A brief introduction to variational auto-encoder model

Consider a dataset consisting of N independent and identically distributed samples, denoted by $\mathbf{x} = \{\mathbf{x}^{(i)}\}_{i=1}^N$, but the true data distribution is unknown. VAE assumes that each sample is randomly generated as follows. First, a latent variable $\mathbf{z}^{(i)}$ is drawn from a prior distribution $p_{\boldsymbol{\theta}^*}(\mathbf{z})$. Subsequently, $\mathbf{x}^{(i)}$ is generated from $p_{\boldsymbol{\theta}^*}(\mathbf{x}|\mathbf{z})$ ^[15]. The prior $p_{\boldsymbol{\theta}^*}(\mathbf{z})$ and the likelihood $p_{\boldsymbol{\theta}^*}(\mathbf{x}|\mathbf{z})$ are supposed to be members of the parametric family of distributions $\{p_{\boldsymbol{\theta}}(\mathbf{z}) : \boldsymbol{\theta} \in \boldsymbol{\Theta}\}$ and $\{p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z}) : \boldsymbol{\theta} \in \boldsymbol{\Theta}\}$ respectively, and $\boldsymbol{\theta}^*$ represents the ground truth^[15]. However, we are still not able to figure out the data distribution because both $\boldsymbol{\theta}^*$ and $\mathbf{z}$ are unknown.

VAE considers the posterior distribution of $\mathbf{z}$,

$$p_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x}) = \frac{p_{\boldsymbol{\theta}}(\mathbf{z})p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z})}{p_{\boldsymbol{\theta}}(\mathbf{x})},$$

which is analytically intractable, and aims to find a standard distribution $q_{\boldsymbol{\lambda}}(\mathbf{z}|\mathbf{x})$ parameterized by $\boldsymbol{\lambda}$ to approximate $p_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x})$ using VI. See a graphical representation of VAE in Figure 3-2. Specially, VAE minimizes the KL divergence between $q_{\boldsymbol{\lambda}}(\mathbf{z}|\mathbf{x})$ and $p_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x})$.

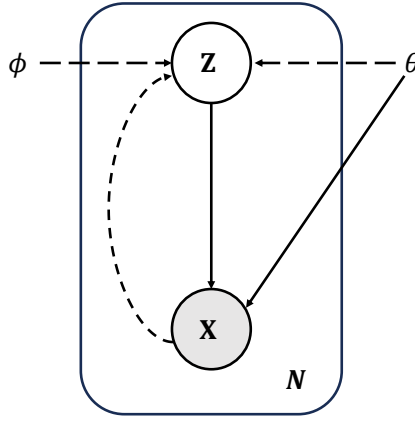


Figure 3-2 Graphical representation of VAE model. Solid lines denote the generative model $p_{\theta}(\mathbf{z})$ and $p_{\theta}(\mathbf{x}|\mathbf{z})$, while the dashed lines denote the variational approximation $q_{\lambda}(\mathbf{z}|\mathbf{x})$ of the intractable posterior $p_{\theta}(\mathbf{z}|\mathbf{x})$ ^[15].

The KL divergence of an individual data point is

$$\begin{aligned} \text{KL}(q_{\lambda}(\mathbf{z}|\mathbf{x}^{(i)})||p_{\theta}(\mathbf{z}|\mathbf{x}^{(i)})) &= -\sum_{\mathbf{z}} q_{\lambda}(\mathbf{z}|\mathbf{x}^{(i)}) \log \frac{p_{\theta}(\mathbf{z}|\mathbf{x}^{(i)})}{q_{\lambda}(\mathbf{z}|\mathbf{x}^{(i)})} \\ &= -\sum_{\mathbf{z}} q_{\lambda}(\mathbf{z}|\mathbf{x}^{(i)}) \log \frac{\frac{p_{\theta}(\mathbf{x}^{(i)}, \mathbf{z})}{p_{\theta}(\mathbf{x}^{(i)})}}{q_{\lambda}(\mathbf{z}|\mathbf{x}^{(i)})} \\ &= -\sum_{\mathbf{z}} q_{\lambda}(\mathbf{z}|\mathbf{x}^{(i)}) [\log \frac{p_{\theta}(\mathbf{x}^{(i)}, \mathbf{z})}{q_{\lambda}(\mathbf{z}|\mathbf{x}^{(i)})} - \log p_{\theta}(\mathbf{x}^{(i)})]. \end{aligned}$$

Then we can obtain

$$\begin{aligned} \log p_{\theta}(\mathbf{x}^{(i)}) &= \text{KL}(q_{\lambda}(\mathbf{z}|\mathbf{x}^{(i)}) || p_{\theta}(\mathbf{z}|\mathbf{x}^{(i)})) + \sum_{\mathbf{z}} q_{\lambda}(\mathbf{z}|\mathbf{x}^{(i)}) \log \frac{p_{\theta}(\mathbf{x}^{(i)}|\mathbf{z})p_{\theta}(\mathbf{z})}{q_{\lambda}(\mathbf{z}|\mathbf{x}^{(i)})} \\ &= \text{KL}(q_{\lambda}(\mathbf{z}|\mathbf{x}^{(i)})||p_{\theta}(\mathbf{z}|\mathbf{x}^{(i)})) + L(\boldsymbol{\theta}, \boldsymbol{\lambda}; \mathbf{x}^{(i)}). \end{aligned}$$

Since the KL divergence is non-negative,

$$\log p_{\theta}(\mathbf{x}^{(i)}) \geq \text{LB}(\boldsymbol{\theta}, \boldsymbol{\lambda}; \mathbf{x}^{(i)}) = \mathbb{E}_{q_{\lambda}(\mathbf{z}|\mathbf{x}^{(i)})} [\log(p_{\theta}(\mathbf{x}^{(i)}, \mathbf{z})) - \log(q_{\lambda}(\mathbf{z}|\mathbf{x}^{(i)}))]. \quad (3-12)$$

As introduced in section 2.2.1, minimizing KL is equivalent to maximizing ELBO, and the ELBO is

$$\begin{aligned} \text{LB}(\boldsymbol{\theta}, \boldsymbol{\lambda}; \mathbf{x}^{(i)}) &= \sum_{\mathbf{z}} q_{\lambda}(\mathbf{z}|\mathbf{x}^{(i)}) \log \left[\frac{p_{\theta}(\mathbf{x}^{(i)}, \mathbf{z})}{q_{\lambda}(\mathbf{z}|\mathbf{x}^{(i)})} \right] \\ &= \mathbb{E}_{q_{\lambda}(\mathbf{z}|\mathbf{x}^{(i)})} [\log(p_{\theta}(\mathbf{x}^{(i)}, \mathbf{z})) - \log(q_{\lambda}(\mathbf{z}|\mathbf{x}^{(i)}))] \end{aligned} \quad (3-13)$$

$$\begin{aligned} &= \sum_{\mathbf{z}} q_{\lambda}(\mathbf{z}|\mathbf{x}^{(i)}) \left\{ \log(p_{\theta}(\mathbf{x}^{(i)}|\mathbf{z})) + \log\left(\frac{p_{\theta}(\mathbf{z})}{q_{\lambda}(\mathbf{z}|\mathbf{x}^{(i)})}\right) \right\} \\ &= \mathbb{E}_{q_{\lambda}(\mathbf{z}|\mathbf{x}^{(i)})} [\log(p_{\theta}(\mathbf{x}^{(i)}|\mathbf{z}))] - \text{KL}[q_{\lambda}(\mathbf{z}|\mathbf{x}^{(i)})||p_{\theta}(\mathbf{z})], \end{aligned} \quad (3-14)$$

where the first term in equation (3-14) is the reconstruction loss and the second term is the regularization loss. The variational parameter λ is jointly learned with the generative model parameter θ in VAE.

The next step is to calculate the derivatives of the ELBO with respect to λ and θ to derive the gradient descent algorithm for optimizing the ELBO. For the gradient with respect to λ , denoting $\log [p_\theta(\mathbf{x}|\mathbf{z})] + \log \left[\frac{p_\theta(\mathbf{z})}{q_\lambda(\mathbf{z}|\mathbf{x})} \right]$ by $f(\mathbf{z})$, we can obtain the Monte Carlo estimator as

$$\begin{aligned} \nabla_\lambda \mathbb{E}_{q_\lambda(\mathbf{z}|\mathbf{x})} [f(\mathbf{z})] &= \mathbb{E}_{q_\lambda} [f(\mathbf{z}) \nabla_{q_\lambda(\mathbf{z}|\mathbf{x})} \log q_\lambda(\mathbf{z}|\mathbf{x})] \\ &\approx \frac{1}{L} \sum_{l=1}^L [f(\mathbf{z}) \nabla_{q_\lambda(\mathbf{z}|\mathbf{x})} \log q_\lambda(\mathbf{z}^{(l)}|\mathbf{x})], \end{aligned}$$

where $\mathbf{z}^{(l)} \sim q_\lambda(\mathbf{z}|\mathbf{x}^{(i)})$.

However, this naive estimator has very high variance which is impractical. To address the issue above, Kingma and Welling (2013) used the reparametrization trick introduced in section 2.2.3.2. Specifically, we obtain a sample $\mathbf{z}$ from $q_\lambda(\mathbf{z}|\mathbf{x})$ by first sampling ϵ from a fixed distribution $p(\epsilon)$ and then applying a differentiable transformation g_λ on ϵ , that is, $\epsilon \sim p(\epsilon)$ and $\mathbf{z} = g_\lambda(\epsilon, \mathbf{x})$. Then $\mathbb{E}_{q_\lambda(\mathbf{z}|\mathbf{x})} [f(\mathbf{z}^{(i)})]$ becomes

$$\mathbb{E}_{q_\lambda(\mathbf{z}|\mathbf{x})} [f(\mathbf{z}^{(i)})] = \mathbb{E}_{p(\epsilon)} [f(g_\lambda(\epsilon, \mathbf{x}^{(i)}))],$$

and the gradient of variational lower bound with respect to λ becomes

$$\begin{aligned} \nabla_\lambda \mathbb{E}_{q_\lambda(\mathbf{z}|\mathbf{x})} [f(\mathbf{z}^{(i)})] &= \nabla_\lambda \mathbb{E}_{p(\epsilon)} [f(g_\lambda(\epsilon, \mathbf{x}^{(i)}))] \\ &= \mathbb{E}_{p(\epsilon)} [\nabla_\lambda f(g_\lambda(\epsilon, \mathbf{x}^{(i)}))] \\ &\approx \frac{1}{L} \sum_{l=1}^L \nabla_\lambda f(g_\lambda(\epsilon^{(i,l)}, \mathbf{x}^{(i)})), \end{aligned}$$

where $\epsilon^{(i,l)} \sim p(\epsilon)$.

We can also apply the reparametrization trick to estimate the ELBO in equation (3-13), that is,

$$\widetilde{\text{LB}}^1(\theta, \lambda; \mathbf{x}^{(i)}) = \frac{1}{L} \sum_{l=1}^L [\log p_\theta(\mathbf{x}^{(i)}, \mathbf{z}^{(i,l)}) - \log q_\lambda(\mathbf{z}^{(i,l)}|\mathbf{x}^{(i)})], \quad (3-15)$$

where $\mathbf{z}^{(i,l)} = g_\lambda(\epsilon^{(i,l)}, \mathbf{x}^{(i)})$, $\epsilon^{(i,l)} \sim p(\epsilon)$, and $\widetilde{\text{LB}}^1(\theta, \lambda; \mathbf{x}^{(i)})$ is called the stochastic gradient Variational Bayes (SGVB) estimator of the ELBO. If $\text{KL}(q_\lambda(\mathbf{z}|\mathbf{x})||p_\theta(\mathbf{z}))$ in equation (3-14) can be derived analytically, such that only the reconstruction loss requires

to be estimated by sampling, then the SGVB estimator for the ELBO becomes

$$\widetilde{\text{LB}}^2(\boldsymbol{\theta}, \boldsymbol{\lambda}; \mathbf{x}^{(i)}) = \frac{1}{L} \sum_{l=1}^L \log [p_{\boldsymbol{\theta}}(\mathbf{x}^{(i)} | \mathbf{z}^{(i,l)})] - \text{KL}(q_{\boldsymbol{\lambda}}(\mathbf{z} | \mathbf{x}^{(i)}) || p_{\boldsymbol{\theta}}(\mathbf{z})), \quad (3-16)$$

where $\mathbf{z}^{(i,l)} = g_{\boldsymbol{\lambda}}(\boldsymbol{\epsilon}^{(i,l)}, \mathbf{x}^{(i)})$ and $\boldsymbol{\epsilon}^{(i,l)} \sim p(\boldsymbol{\epsilon})$, typically has smaller variance.

Moreover, we can construct an estimator of the ELBO of the full dataset using the minibatches, i.e.,

$$\text{LB}(\boldsymbol{\theta}, \boldsymbol{\lambda}; \mathbf{x}) \simeq \widetilde{\text{LB}}^M(\boldsymbol{\theta}, \boldsymbol{\lambda}; \mathbf{x}^M) = \frac{N}{M} \sum_{i=1}^M \widetilde{\text{LB}}(\boldsymbol{\theta}, \boldsymbol{\lambda}; \mathbf{x}^{(i)}), \quad (3-17)$$

where the $\mathbf{x}^M = \{\mathbf{x}^{(i)}\}_{i=1}^M$ is a subset randomly drawn from the full dataset.

In VAE, the prior distribution for $\mathbf{z}$ is typically chosen as $p_{\boldsymbol{\theta}}(\mathbf{z}) = N(\mathbf{z} | \mathbf{0}, \mathbf{I})$, and $p_{\boldsymbol{\theta}}(\mathbf{x} | \mathbf{z})$ is specified as either a multivariate Gaussian (for real-valued data) or a Bernoulli distribution (for binary data), with parameters derived from $\mathbf{z}$ via a multi-layer perceptron (MLP, a fully-connected neural network). The variational approximate distribution is assumed to be a multivariate Gaussian with diagonal covariance structure, i.e.,

$$\log q_{\boldsymbol{\lambda}}(\mathbf{z} | \mathbf{x}^{(i)}) = \log N(\mathbf{z}; \boldsymbol{\mu}^{(i)}, \boldsymbol{\sigma}^{2(i)} \mathbf{I}).$$

Here, $\boldsymbol{\mu}^{(i)}$ and $\boldsymbol{\sigma}^{(i)}$ are outputs of an MLP, in fact nonlinear functions of the data point $\mathbf{x}^{(i)}$ and the variational parameters $\boldsymbol{\lambda}$. With the specification above, the KL divergence in equation (3-14) can be computed analytically. Thus, the estimator of the ELBO becomes

$$\text{LB}(\boldsymbol{\theta}, \boldsymbol{\lambda}; \mathbf{x}^{(i)}) \simeq \frac{1}{2} \sum_{j=1}^J (1 + \log((\sigma_j^{(i)})^2) - (\mu_j^{(i)})^2 - (\sigma_j^{(i)})^2) + \frac{1}{L} \sum_{l=1}^L \log p_{\boldsymbol{\theta}}(\mathbf{x}^{(i)} | \mathbf{z}^{(i,l)}), \quad (3-18)$$

where $\mathbf{z}^{(i,l)} = \boldsymbol{\mu}^{(i)} + \boldsymbol{\sigma}^{(i)} \odot \boldsymbol{\epsilon}^{(l)}$ and $\boldsymbol{\epsilon}^{(l)} \sim N(\mathbf{0}, \mathbf{I})$.

With the reparameterization trick applied in VAE, the Monte Carlo estimate of the ELBO is differentiable with respect to $\boldsymbol{\lambda}$ ^[15], and the back-propagation technique in neural networks enables the effective training of model parameters $\boldsymbol{\lambda}$ and $\boldsymbol{\theta}$.

As introduced in Section 2.3, the variational approximation $q_{\boldsymbol{\lambda}}(\mathbf{z} | \mathbf{x})$ is called the encoder while the likelihood $p_{\boldsymbol{\theta}}(\mathbf{x} | \mathbf{z})$ is the decoder. Note that the encoder and decoder in VAE are probabilistic. VAE generates new data that approximately follows the true distribution by first generating a code $\mathbf{z}$ from the encoder and given the code, generating a sample from the decoder.

3.4.2 A VAE-assisted MCMC sampler

Standard MCMC methods using local proposal distributions, which are centered around the current state, mix poorly when sampling from the multi-modal target distributions with distant modes, since these methods can hardly jump over the low-density region between distant modes and typically end up getting stuck in one mode. Therefore, novel MCMC algorithms which can efficiently jump between modes are required for sampling the multi-modal distributions.

Inspired by Gabie et al. (2021) and Hunt-Smith et al. (2024)^[39-40], here we propose a novel MCMC sampler called “VAE-MH”, which combines the Gaussian random-walk MH algorithm with the VAE model. Specifically, we first train the VAE model to produce a new proposal distribution, use this proposal to produce a new MH algorithm, and switch between the new algorithm and the original random-walk MH within the whole algorithm. Note that in this process we constantly re-train the VAE model to update the new proposal so that the corresponding MH algorithm becomes better and better in performance. The primary advantage of this new MCMC sampler is the global nature of its proposed values so that it is not easy for the algorithm to get stuck in a local area. We now give a detailed description of how to combine the VAE model with the random-walk MH to produce the new MH algorithm.

To train the VAE model, we typically use the samples $\{\boldsymbol{\theta}^{(t)}\}_{t=1}^{T_0}$ that generated by the preliminary runs of the Gaussian random-walk MH. However, for special cases, such as multi-modal target distributions with distant modes, Gaussian random-walk MH may fail to collect the information about all the modes. In such cases, we can first use some optimization methods to identify the approximate locations of all the modes, run multiple Gaussian random-walk MH preliminary chains initiated at these different modes respectively, and use the samples generated by all the chains to train the VAE model. Recall the training process of VAE introduced in section 3.4.1. Here the target variable $\boldsymbol{\theta}$ is equivalent to $\mathbf{x}$ in Section 3.4.1. We specify the prior of the latent variable $\mathbf{z}$, $p(\mathbf{z})$, as $N(\mathbf{z}|\mathbf{0}, \mathbf{I})$, and the “likelihood” $p_{\xi}(\boldsymbol{\theta}|\mathbf{z})$ as a multivariate Gaussian distribution whose parameters are computed from $\mathbf{z}$ using an MLP. In addition, we assume the variational approximation distribution for the intractable posterior $p_{\xi}(\mathbf{z}|\boldsymbol{\theta})$ is a multivariate Gaussian, i.e., $q_{\lambda}(\mathbf{z}|\boldsymbol{\theta}^{(t)}) = \log N(\mathbf{z}|\boldsymbol{\mu}^{(t)}, \boldsymbol{\sigma}^{2(t)}\mathbf{I})$, where the mean and standard deviations are computed from the $\boldsymbol{\theta}^{(t)}$ with the MLP. The objective function is defined by equation (3-17), and the corresponding estimator $\widetilde{\text{LB}}(\boldsymbol{\xi}, \boldsymbol{\lambda}; \boldsymbol{\theta}^{(t)})$ is provided in equation (3-18).

Now we describe how to use the trained VAE model to construct a proposal distribution. Recall that VAE generates a new θ sample by first drawing $\mathbf{z}$ from $q_{\lambda}(\mathbf{z}|\theta)$, which is estimated using $\{\theta^{(t)}\}_{t=1}^{T_0}$, and then drawing θ given $\mathbf{z}$ from $p_{\xi}(\theta|\mathbf{z})$. Each θ generated this way is independent of the previous generated sample since each $\mathbf{z}^{(i)}$ is generated independently from $q(\mathbf{z}|\theta)$. We generate K samples of θ , $\{\theta^{(k)}\}_{k=1}^K$ and use them to estimate a proposal distribution $Q(\theta)$.

Suppose that θ is a d -dimensional variable, i.e., $\theta = (\theta_1, \theta_2, \dots, \theta_d)^\top$, and we use the following decomposition to construct the d -dimensional distribution $Q(\theta)$ by estimating d 1-dimensional distributions:

$$Q(\theta) = Q(\theta_1) \times Q(\theta_2 | \theta_1) \times Q(\theta_3 | \theta_2, \theta_1) \times \dots \times Q(\theta_d | \theta_1, \theta_2, \theta_3, \dots, \theta_{d-1}). \quad (3-19)$$

We first use $\{\theta^{(k)}\}_{k=1}^K$ to estimate the marginal distribution of θ_1 , $Q(\theta_1)$. Specifically, we divide the range of θ_1 into b bins, and use the proportion of samples with θ_1 values falls in a particular bin as the probability for that bin. Next, we fix the value of θ_1 and proceed to estimate the conditional distribution $Q(\theta_2 | \theta_1)$ using the same method as for estimating $Q(\theta_1)$. We continue this process until we obtain the last conditional distribution $Q(\theta_d | \theta_1, \theta_2, \theta_3, \dots, \theta_{d-1})$.

This approach was primarily chosen to reduce computational cost and circumvent the necessity for constructing high-dimensional histograms. The number of bins, b , is calculated using the Sturges' Rule, that is,

$$b = 1 + \log_2 K,$$

which is a simple method for determining the number of bins based on the number of available data points.

The estimated $Q(\theta)$ is a new proposal distribution. A new value θ' is generated by randomly selecting one sample from $\{\theta^{(k)}\}_{k=1}^K$, which is used to estimate the proposal distribution. Note that θ' generated this way does not depend on the current state of the algorithm, θ . Thus the new proposal $Q(\theta)$ is an independent one. By using $Q(\theta)$ as the proposal distribution, we obtain a new independent MH algorithm.

In each iteration of the whole algorithm, we use a certain probability denoted by p_{VAE} to decide if we implement the new independent MH or the original random-walk MH. At this stage, we tuning p_{VAE} manually. Note that we retrain the VAE model every τ iterations using the last 4000 samples accumulated by then together with the initial samples used to train the VAE model. Since the distribution of the training samples gets closer and closer

to the true target distribution, the proposal distribution constructed by the VAE model will be a more accurate approximation of the target distribution and generate global proposals which can easily jump to any region with high density in the parameter space.

The procedure of the complete new VAE-MH algorithm is described in Algorithm 3.5 below. In our experiments of Section 4.3, when training the VAE model, we use the default stochastic gradient ascent method in PyTorch, which is Adam, and choose the learning rate to be 0.001. We set the mini-batch size for estimating the ELBO to be $M = 128$, and the number of ϵ samples to estimate the ELBO to be $L = 1$. For the MCMC algorithm, we set the step-size of the Gaussian random-walk MH to be $\sigma_{\text{MH}} = 0.5$, the VAE-retraining time to be $\tau = 1000$, the total number of iterations to be $N = 10^5$, and the number of bins for estimating the VAE proposal to be $b = 20$. The probability p_{VAE} is set to be 0.2 in the examples for the mixture of 2-dimensional Gaussian and mixture of 2-dimensional Student t distributions, and 0.5 in the example for the mixture of 10-dimensional Gaussian.

Algorithm 3.5 A VAE-assisted MCMC sampler

```

1: For  $i = 1, 2, \dots, N$  do,
2:   Generate  $u \sim U(0, 1)$ .
3:   If  $u < p_{\text{VAE}}$  then
4:     MH algorithm with a proposal function from a VAE model,
5:   else
6:     MH algorithm with Gaussian proposal,
7:   end if
8:   If  $i \bmod \tau = 0$  then
9:     Retrain VAE model using the samples obtained from MCMC sampling and some prior
       training samples.
10:  end if
11: end for

```

3.5 Performance Evaluation Criteria for Algorithms

Efficiency

Given the target distribution $p(\theta)$ and the corresponding MCMC algorithm to sample from it, in order to estimate the expectation of a p -integrable function of θ , i.e., $\int f(\theta)p(\theta)d\theta$, we use the mean calculated by the samples $\theta^{(1)}, \theta^{(2)}, \dots, \theta^{(T)}$ generated by the MCMC algorithm, i.e. $\hat{p}(f) = \frac{1}{T} \sum_{t=1}^T f(\theta^{(t)})$. The efficiency of this estimate is defined by

$$E = \frac{v_f}{v} = \frac{1}{1 + 2 \sum_{k=1}^{\infty} \rho_k},$$

where $\rho_k = \text{Cor}[f(\theta^{(t)}), f(\theta^{(t+k)})]$ is the lag- k auto-correlation of the chain

$\{f(\theta^{(t)})\}_{t=1}^{\infty}$, v_f is the true variance of $f(\theta)$, and v is the asymptotic variance of $\sqrt{T}\hat{p}(f)$ ^[20]. An equivalent quantity to E is effective sample size (ESS), which is calculated as TE . The ESS is the number of independent samples that are equivalent to the correlated samples obtained with the MCMC algorithm. In the numerical studies of Chapter 4, we focus on the efficiency for estimating θ , that is, $f(\theta) = \theta$.

Lag-1 auto-correlation

For an MCMC algorithm, as introduced above, ρ_1 is the lag-1 auto-correlation of the generated Markov chain, and it is also an indicator for the efficiency of the algorithm. A value of ρ_1 close to 0 indicates that the states are nearly independent, while high values of ρ_1 suggest strong correlation between states, implying that the chain may require more iterations before reaching equilibrium.

Average acceptance rate

For an MH algorithm with the target distribution $p(\theta)$, a new state θ^* is generated from the proposal distribution $Q(\theta^*|\theta)$ given the current state θ . The algorithm accepts θ^* as the next iteration with probability

$$\alpha(\theta^*|\theta) = \min \left\{ 1, \frac{p(\theta^*)Q(\theta|\theta^*)}{p(\theta)Q(\theta^*|\theta)} \right\}.$$

The average acceptance rate P_{jump} is defined by

$$P_{\text{jump}} = \int \int p(\theta) \alpha(\theta^*|\theta) Q(\theta^*|\theta) d\theta^* d\theta.$$

The average acceptance rate P_{jump} is the quantity that we inspect when tuning the step-size parameter in an MH algorithm. For Gaussian random-walk MH, Roberts et al. (1997)^[41] showed that the optimal P_{jump} is approximately 0.234 for high-dimensional distributions and the corresponding optimal step size is $2.38/\sqrt{d}$, where d is the dimension. In practice, the result applied to even low-dimensional cases with $d \geq 5$, while for $d = 1$, the optimal P_{jump} is around 0.44. Furthermore, if the target distribution is multivariate with covariance matrix Σ and we can obtain an estimate of it, i.e., $\hat{\Sigma}$, then the optimal random-walk proposal distribution becomes $(\theta^*|\theta) \sim N_d(\theta, 2.38^2 \hat{\Sigma}/d)$ ^[41]. For the MALA algorithm, Roberts et al. (1998)^[43] showed that the optimal asymptotic P_{jump} is approximately 0.574 and the optimal step size can be calculated as $1.65/d^{1/6}$. For both random-walk MH and MALA, we can use adaptive methods to find the step sizes that achieve the optimal P_{jump} .

CHAPTER 4 NUMERICAL STUDIES

4.1 Compare the Speed and Accuracy of Estimating Gaussian Target via MCMC and GVI

In this experiment, the target distributions are Gaussian $N(\mathbf{0}, \mathbf{I}_d)$ with dimensions $d = 2, 10, 50, 100, 500$ respectively, and we use both MCMC and GVI with factor decomposed covariance, i.e., FGVl to approximate each of the distributions. The MCMC algorithm we employ here is Metropolis-within-Gibbs algorithm. In one iteration of the algorithm, each coordinate θ_j is updated using the random-walk MH given the current values of the other coordinates, and all the coordinates are updated in turn. The proposal distribution for updating θ_j is $\theta_j^* \mid \theta_j \sim N(\theta_j, s_j^2)$, for $j = 1, 2, \dots, d$, and we tune the s_j here to make the average acceptance rate for each coordinate around 40%. The FGVl we use here has $f = 1$ and the setting of tuning parameters for the FGVl algorithm is introduced in Section 3.2.2.2.

Then we have two estimates of the true mean and covariance matrix of the target distribution. The first is the mean and covariance calculated using the MCMC samples and the second is the mean and covariance of the FGVl approximation distribution. The accuracy of the estimates is measured using the root mean squared error (RMSE) between the true values and the estimated values. RMSE is calculated using the formula,

$$\text{RMSE} = \sqrt{\frac{\sum_{i=1}^k (\lambda_i - \hat{\lambda}_i)^2}{k}},$$

where $k = d + d + d \times (d - 1)/2$ is the total number of free parameters in a Gaussian distribution of dimension d , because there are d mean parameters, d standard deviation parameters and $d \times (d - 1)/2$ free correlation parameters, λ is a k -dimensional vector containing the true values of all these mean, standard deviation and correlation parameters and $\hat{\lambda}$ is the vector containing the corresponding estimates. The smaller is the RMSE, the higher is the estimation accuracy of the corresponding algorithm.

We record the running time of FGVl and run the Metropolis-within-Gibbs algorithm for nearly the same amount of time as FGVl. The RMSE and running time of both FGVl and Metropolis-within-Gibbs for each Gaussian target distribution are shown in table 4-1.

From the results, we find that the moment estimates obtained from the MCMC algo-

rithm are much less accurate than those from FGVI and the problem gets more severe as the dimension of the target distribution increases. However, the estimation accuracy of FGVI remains stable as the dimension goes up.

Table 4-1 Estimation accuracy of Metropolis-within-Gibbs and FGVI for the moments of standard Gaussian distributions with $d = 2, 10, 50, 100, 500$.

d	RMSE(FGVI)	RMSE(MCMC)	Time(s)
2	0.0018	0.0222	0.6335
10	0.0010	0.0293	0.9980
50	0.0006	0.0526	4.0623
100	0.0006	0.0602	23.2442
500	0.0001	0.0747	4162.039

4.2 Linear Transformations in HMC via GVI

4.2.1 Posterior sampling in logistic regression models

4.2.1.1 The model setup

Here, we first give a brief introduction to the binary logistic regression model. Let $\mathbf{y} = (y_1, \dots, y_n)^\top$ be a vector of the observed binary responses, and $\mathbf{X}$ the corresponding design matrix, each row of which is $\mathbf{x}_i = (1, x_{i2}, \dots, x_{id})^\top$, for $i = 1, 2, \dots, n$. All the predictors have been standardized to improve the mixing^[49]. We consider the following generalized linear model,

$$(y_i | p_i) \stackrel{ind}{\sim} \text{Bernoulli}(p_i),$$

$$p_i = g(\eta_i),$$

$$\eta_i = \beta_1 x_{i1} + \dots + \beta_p x_{id},$$

where $g(\cdot)$ is the inverse logit transform, i.e., $g(\eta_i) = \frac{e^{\eta_i}}{1+e^{\eta_i}}$. Then we have a logistic regression model as follows,

$$\log \frac{p(\mathbf{x}_i)}{1 - p(\mathbf{x}_i)} = \beta_1 + \beta_2 x_{i2} + \dots + \beta_d x_{id} = \boldsymbol{\beta}^\top \mathbf{x}_i.$$

Therefore, the likelihood function is

$$L(\boldsymbol{\beta}) = \prod_{i=1}^n \left[p(\mathbf{x}_i)^{y_i} (1 - p(\mathbf{x}_i))^{1-y_i} \right],$$

and the corresponding log-likelihood function is

$$\begin{aligned}
 l(\boldsymbol{\beta}) &= \sum_{i=1}^n [y_i \log p(\mathbf{x}_i) + (1 - y_i) \log (1 - p(\mathbf{x}_i))] \\
 &= \sum_{i=1}^n \left\{ y_i \log \frac{p(\mathbf{x}_i)}{1 - p(\mathbf{x}_i)} + \log [1 - p(\mathbf{x}_i)] \right\} \\
 &= \sum_{i=1}^n [y_i \boldsymbol{\beta}^\top \mathbf{x}_i - \log (1 + e^{\boldsymbol{\beta}^\top \mathbf{x}_i})].
 \end{aligned}$$

The partial derivative of the log-likelihood with respect to each β_j is

$$\begin{aligned}
 \frac{\partial l(\boldsymbol{\beta})}{\partial \beta_j} &= \sum_{i=1}^n [y_i x_{ij} + \frac{1}{1 + e^{\boldsymbol{\beta}^\top \mathbf{x}_i}} e^{\boldsymbol{\beta}^\top \mathbf{x}_i} x_{ij}] \\
 &= \sum_{i=1}^n \left[(y_i - \frac{1}{1 + e^{-\boldsymbol{\beta}^\top \mathbf{x}_i}}) x_{ij} \right],
 \end{aligned}$$

which will be used when implementing the GVI, MALA and HMC algorithms later on.

This experiment aims to estimate the parameter vector $\boldsymbol{\beta} = (\beta_1, \dots, \beta_d)^\top$ under Bayesian framework, and use MCMC methods to sample the posterior distribution. Here we specify to $\boldsymbol{\beta}$ a vague prior $p(\boldsymbol{\beta})$, which is centered at 0, namely, $\boldsymbol{\beta} \sim N_d(\mathbf{0}, 100\mathbf{I}_d)$. Then the posterior distribution of $\boldsymbol{\beta}$ is

$$p(\boldsymbol{\beta} \mid \mathbf{y}) \propto l(\boldsymbol{\beta}) \times p(\boldsymbol{\beta}).$$

4.2.1.2 The results of numerical studies

To sample from the posterior distribution of the Bayesian logistical regression model, we compare the performance of five algorithms, which are random-walk MH with the scale matrix estimated using the burn-in samples (RMH), AM, preconditioned MALA with the preconditioning matrix estimated using the burn-in samples, HMC with linear transformation matrix estimated using the burn-in samples and HMC with linear transformation matrix estimated using GVI. All of RMH, AM, MALA and HMC are introduced in Section 3.1, and HMC with linear transformation is described in Section 3.3. For each algorithm, we tune the hyper-parameters according to the rules introduced in Section 3.5. Two datasets are used in this study. The first is the Pima Indian dataset with $d = 8$ and $n = 532$ available in the R package MASS, and the second is the German credit dataset with $d = 24$ and $n = 1000$ obtained from the UCL machine learning repository. When

analysing each dataset, we use all the five algorithms and for each algorithm, run a chain of 10^6 plus a burn-in of 10^4 iterations.

Since that the dimensions of data is not large, we use the CGVI introduced in section 3.2.2.1 to derive a Gaussian approximation of the target distribution, and use the covariance matrix of the approximation distribution for linear transformation in HMC. The running times of CGVI are 4.81s and 11.16s for the Pima Indian and German credit datasets respectively, and comparable to the running times of the burn-in for HMC, which are 4.61s and 11.69s for the two datasets respectively.

Table 4-2 Comparison of five MCMC algorithms on Pima Indian dataset with $d=8$, $n=532$.

Method	Time(s)	E	P_{jump}	ρ_1
RMH	64.9021	0.0382	0.2394	0.9098
AM	113.9103	0.0386	0.2493	0.9211
MALA	146.6414	0.3012	0.5778	0.5467
HMC	359.0816	0.8630	0.9937	0.0623
HMC-GVI	356.7121	0.9200	0.9935	0.0097

Table 4-3 Comparison of five MCMC algorithms on German credit dataset with $d=24$, $n=1000$.

Method	Time(s)	E	P_{jump}	ρ_1
RMH	137.9467	0.0122	0.2371	0.9590
AM	186.7112	0.0138	0.2277	0.9606
MALA	298.3795	0.1714	0.5383	0.7901
HMC	985.9431	0.8422	0.9888	0.0925
HMC-GVI	992.9441	1.0009	0.9871	0.0119

The running time, efficiency (E), average acceptance rate (P_{jump}) and lag-1 autocorrelation (ρ_1) are shown for each algorithm. Note that hereafter, the efficiency and lag-1 autocorrelation are all averages over all the parameters in the target distribution. The results for the Pima Indian dataset are presented in table 4-2 and those for the German credit dataset are in table 4-3. From the results we can see that GVI provides more accuracy when estimating the covariance matrix in a similar amount of running time as HMC, and this leads to enhanced efficiency of the corresponding algorithm, as verified by $E = 0.9200$ and 1.0009 for the HMC with linear transformation via GVI, compared to $E = 0.8630$ and 0.8422 for the HMC with linear transformation using the burn-in samples.

In addition, the trace and autocorrelation function (ACF) plots of β_1 in the Pima Indian dataset for each of the five algorithms are given in figure 4-1. From the plots, we find that MALA, HMC and HMC with linear transformation via GVI perform much better than the RMH and AM since their chains mix faster and the autocorrelations drop to 0 more quickly. HMC with linear transformation via GVI shows even more improvement compared to MALA and HMC.

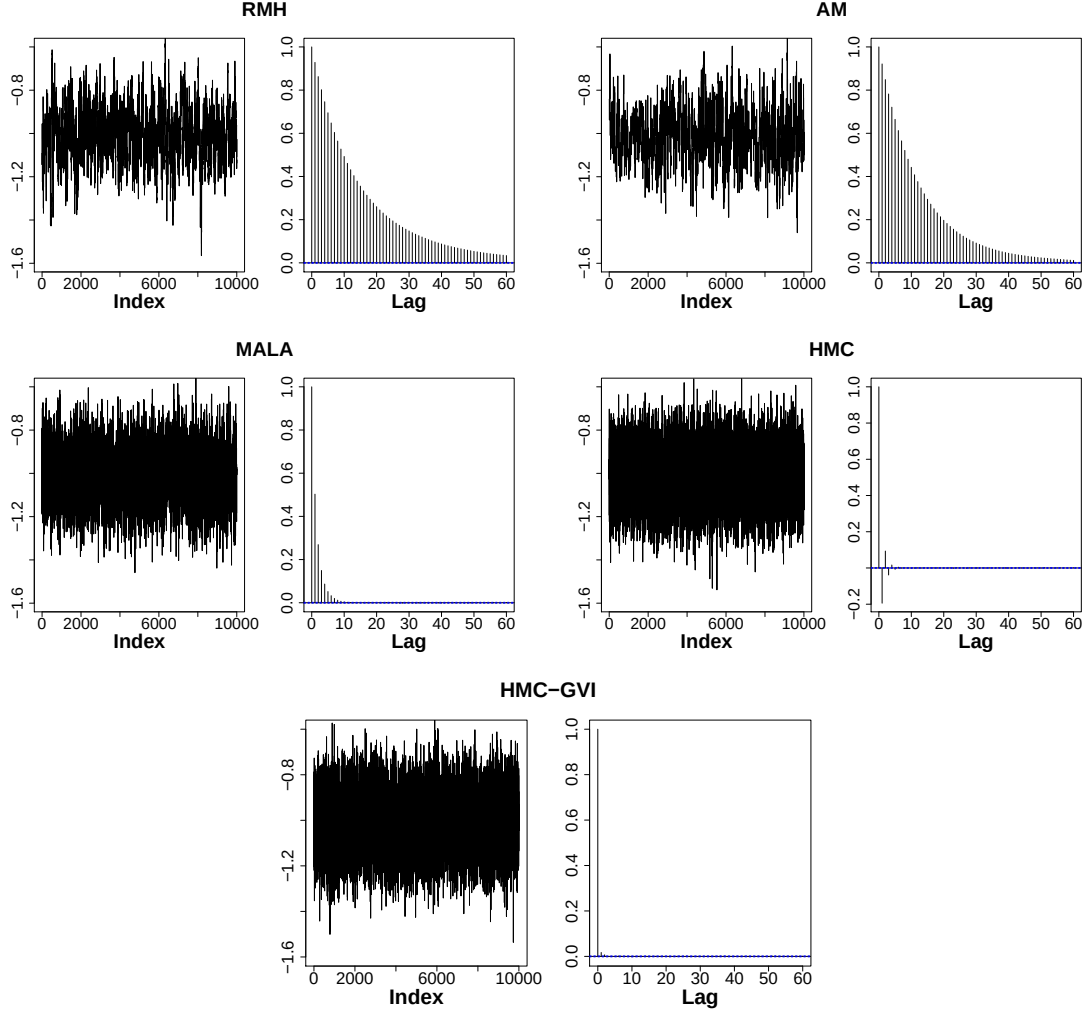


Figure 4-1 Trace and ACF plots of β_1 in the Pima Indian dataset for each of the five algorithms.

4.2.2 Sampling from a 100-dimensional Gaussian distribution

4.2.2.1 The model setup

In this experiment, the target distribution is a 100-dimensional multivariate Gaussian distribution with mean zero and covariance matrix Σ , that is,

$$p(\theta) \propto \exp\left\{-\frac{1}{2}\theta^\top \Sigma^{-1}\theta\right\}.$$

The diagonal elements of Σ are generated from a Gamma distribution with the shape parameter equal to 2 and the scale parameter equal to 3, and all the off-diagonal elements are fixed at 0.8. Then the log-likelihood function is

$$l(\theta) = -\frac{1}{2}\theta^\top \Sigma^{-1}\theta,$$

and its gradient respect to θ is

$$\frac{\partial l(\theta)}{\partial \theta} = -\Sigma^{-1}\theta,$$

which will be used later when implementing the GVI, MALA and HMC algorithms.

4.2.2.2 The simulation results

In this experiment, we also compare the five MCMC algorithms mentioned in Section 4.2.1 for sampling the 100-dimensional Gaussian target of Section 4.2.2.1. For each algorithm, we run a chain of 10^6 plus a burn-in of 10^4 iterations. For the HMC algorithm with linear transformation via GVI, we employ the FGVI with $f = 5$ to compute the covariance matrix. Hyper-parameters for all the MCMC algorithms are tuned according to rules described in Section 3.5. The running time of FGVI is 23.87s, and that of the burn-in for HMC is 57.20s. The running time, efficiency (E), average acceptance rate (P_{jump}) and lag-1 autocorrelation (ρ_1) are shown for each algorithm in table 4-4. Each result is an average over 3 replicated runs.

Table 4-4 Comparison of five MCMC algorithms for sampling the 100-dimensional Gaussian distribution.

Method	Time(s)	E	P_{jump}	ρ_1
RMH	1660.89	0.0027	0.9383	0.9984
AM	2075.81	0.0021	0.3100	0.9963
MALA	1031.212	0.0274	0.5657	0.8213
HMC	5657.62	0.9438	0.9744	0.00345
HMC-GVI	5496.92	1.0050	0.9745	-0.0053

The results in table 4-4 show that for high-dimensional target distributions, the running time of FGVI is significantly shorter than that of the burn-in for HMC. However, FGVI provides a more accurate estimate of the covariance matrix than the burn-in samples of MCMC, and this improved accuracy leads to an MCMC algorithm with higher efficiency, as demonstrated by that fact that $E = 1.0050$ for the HMC with linear trans-

formation via FGVI while $E = 0.9438$ for the HMC with transformation matrix estimated by the burn-in samples.

4.2.3 Posterior sampling in a 509-dimensional GLMM

4.2.3.1 The model setup

The generalized linear mixed model (GLMM) is an extension to the generalized linear model in that the linear predictors of GLMM contains both fixed effects and random effects. The set of response variables for the i th subject is $\mathbf{y}_i = (y_{i1}, \dots, y_{in_i})^\top$, which is distributed according to the exponential family with the expectation related to the linear predictor via a smooth invertible link function $g(\cdot)^{[14]}$, i.e.,

$$\mu_{ij} = \mathbb{E}(y_{ij}),$$

$$g(\mu_{ij}) = \mathbf{X}_{ij}^\top \boldsymbol{\beta} + \mathbf{Z}_{ij}^\top b_i, \text{ for } i = 1, \dots, n, j = 1, \dots, n_i,$$

$$b_i \sim N(\mathbf{0}, \mathbf{G}),$$

$$\boldsymbol{\beta} \sim N(\mathbf{0}, \sigma_\beta^2 \mathbf{I}).$$

Here $\mathbf{X}$ and $\boldsymbol{\beta}$ are the fixed-effects design matrix and parameter vector respectively; $\mathbf{Z}$ and b_i are the random-effects design matrix and the random effect for the i th subject respectively. Here we consider binary responses, where $y_{ij} \sim \text{Bernoulli}(\mu_{ij})$ with the logit link function $g(\mu_{ij}) = \log \frac{\mu_{ij}}{1-\mu_{ij}}$. Let $\mathbf{G} = \mathbf{L}\mathbf{L}^\top$, where $\mathbf{L}$ is the Cholesky factor of $\mathbf{G}$, and let $\mathbf{L}^*$ denote the matrix for which $\mathbf{L}_{ii}^* = \log(\mathbf{L}_{ii})$, $\mathbf{L}_{ij}^* = \mathbf{L}_{ij}$ if $i \neq j$ and $\boldsymbol{\xi} = \text{vech}(\mathbf{L}^*)$. Here we assume a normal prior for $\boldsymbol{\xi}$, that is, $\boldsymbol{\xi} \sim N(\mathbf{0}, \sigma_\xi^2 \mathbf{I})^{[14]}$. The joint posterior distribution can be written as

$$p(\mathbf{y}, \boldsymbol{\theta}) \propto h(\boldsymbol{\theta}) = p(\boldsymbol{\beta})p(\boldsymbol{\xi}) \times \prod_{i=1}^n \left\{ p(b_i | \boldsymbol{\xi}) \prod_{j=1}^{n_i} p(y_{ij} | \boldsymbol{\beta}, b_i) \right\},$$

and the log-posterior is

$$\begin{aligned} \log h(\boldsymbol{\theta}) &= \sum_{i=1}^n \sum_{j=1}^{n_i} \log p(y_{ij} | \boldsymbol{\beta}, b_i) + \sum_{i=1}^n \log p(b_i | \boldsymbol{\xi}) + \log p(\boldsymbol{\beta}) + \log p(\boldsymbol{\xi}) \\ &= \sum_{i,j} \left\{ y_{ij} \left(\mathbf{X}_{ij}^\top \boldsymbol{\beta} + \mathbf{Z}_{ij}^\top b_i \right) - h_1 \left(\mathbf{X}_{ij}^\top \boldsymbol{\beta} + \mathbf{Z}_{ij}^\top b_i \right) \right\} \\ &\quad - n \log |\mathbf{L}| - \frac{1}{2} \sum_{i=1}^n b_i^\top \mathbf{L}^{-\top} \mathbf{L}^{-1} b_i - \frac{1}{2\sigma_\beta^2} \boldsymbol{\beta}^\top \boldsymbol{\beta} - \frac{1}{2\sigma_\xi^2} \boldsymbol{\xi}^\top \boldsymbol{\xi} + C, \end{aligned} \quad (4-1)$$

where C is a constant not depending on $\boldsymbol{\theta}$. For the logistic GLMM, we have $h_1(x) = \log\{1 + \exp(x)\}$. The gradient of $\log h(\boldsymbol{\theta})$, to be used later, is $[\nabla_{b_1} \log h(\boldsymbol{\theta}), \dots, \nabla_{b_n} \log h(\boldsymbol{\theta}), \nabla_{\boldsymbol{\beta}} \log h(\boldsymbol{\theta}), \nabla_{\boldsymbol{\xi}} \log h(\boldsymbol{\theta})]^{[14]}$, where

$$\nabla_{b_i} \log h(\boldsymbol{\theta}) = \sum_{j=1}^{n_i} \{y_{ij} - h'_1(X_{ij}^\top \boldsymbol{\beta} + Z_{ij}^\top b_i)\} Z_{ij} - \mathbf{L}^{-\top} \mathbf{L}^{-1} b_i, \text{ for } i = 1, \dots, n,$$

$$\nabla_{\boldsymbol{\beta}} \log h(\boldsymbol{\theta}) = \sum_{i=1}^n \sum_{j=1}^{n_i} \{y_{ij} - h'_1(X_{ij}^\top \boldsymbol{\beta} + Z_{ij}^\top b_i)\} X_{ij} - \frac{1}{\sigma_{\boldsymbol{\beta}}^2} \boldsymbol{\beta},$$

$$\nabla_{\boldsymbol{\xi}} \log h(\boldsymbol{\theta}) = -n \mathbf{1}_{\text{diag}(\mathbf{L})} + \mathbf{1}_{\boldsymbol{\xi}} \odot \text{vech}(\mathbf{A}) - \frac{1}{\sigma_{\boldsymbol{\xi}}^2} \boldsymbol{\xi},$$

with $\mathbf{A} = \sum_{i=1}^n \mathbf{L}^{-\top} \mathbf{L}^{-1} b_i b_i^\top \mathbf{L}^{-\top}$. The i -th element of the vector $\mathbf{1}_{\boldsymbol{\xi}}$ is $\exp(\xi_i)$ if ξ_i corresponds to a diagonal element of $\mathbf{L}$ and 1 otherwise, and the i th element of the vector $\mathbf{1}_{\text{diag}(\mathbf{L})}$ is 1 if the i th elements of $\boldsymbol{\xi}$ corresponds to a diagonal element of $\mathbf{L}$ and 0 otherwise.

The data used in this study is the polypharmacy dataset available in the R package `aplore3`. Following Tan et al. (2018)^[14], we consider the following GLMM,

$$\begin{aligned} \text{logit}(\mu_{ij}) = & \beta_0 + \beta_1 \text{Gender}_i + \beta_2 \text{Race}_i + \beta_3 \text{Age}_{ij} + \beta_4 \text{MHV1}_{ij} + \\ & \beta_5 \text{MHV2}_{ij} + \beta_6 \text{MHV3}_{ij} + \beta_7 \text{INPTMHV}_{ij} + b_i, \end{aligned}$$

where $i = 1, \dots, 500, j = 1, \dots, 7$. The response is whether the subject is using drugs from 3 or more distinct groups. The predictors include Gender (1 for male and 0 for female), race (0 for white and 1 for other races), and age. Binary indicators for the number of outpatient or inpatient mental health visits are also included as predictors, with MHV1 = 1 if the subject had between 1 and 5 outpatient visits, MHV2 = 1 if 6 to 14, MHV3 = 1 if 15 or more, and INPTMHV = 0 if there were no inpatient mental health visits and 1 otherwise. Write $\boldsymbol{\beta} = (\beta_0, \beta_{\text{gender}}, \beta_{\text{race}}, \beta_{\text{age}}, \beta_{M1}, \beta_{M2}, \beta_{M3}, \beta_{IM})^\top$, $\mathbf{b} = (b_1, \dots, b_{500})^\top$, and then the parameters are $\boldsymbol{\theta} = (\boldsymbol{\beta}^\top, \mathbf{b}^\top, \boldsymbol{\xi}^\top)^\top$. Note that for this example, $\boldsymbol{\xi}$ is a scalar, so that the dimension of $\boldsymbol{\theta}$ is 509. The prior distributions we assign to the parameters are $p(\boldsymbol{\beta}) = N(\mathbf{0}, 100\mathbf{I}_8)$, $p(\boldsymbol{\xi}) = N(0, 100)$ and $p(b_i | \boldsymbol{\xi}) = N(0, \exp(2\xi))$. The posterior distribution is as in equation (4-1).

4.2.3.2 The results of numerical studies

To sample from the posterior distribution of the GLMM model for the polypharmacy dataset, we compare the performance of four algorithms, which are preconditioned MALA

with the preconditioning matrix estimated using the preliminary MALA samples, preconditioned MALA with the preconditioning matrix estimated using GVI, HMC with linear transformation matrix estimated using the preliminary HMC samples and HMC with linear transformation matrix estimated using GVI. We do not consider the random-walk MH and AM algorithms here because they are both too low in efficiency. For each algorithm, we tune the hyper-parameters according to the rules introduced in Section 3.5, and run a chain of 10^6 plus a burn-in of 10^4 .

Due to the complex nature of the target distribution in this experiment, we use the GVI with sparse precision matrix proposed by Tan et al. (2018)^[14] to estimate the covariance matrix which is used in both the MALA and HMC algorithms. This GVI method is described in Section 3.2.2.3. For the polypharmacy dataset, the running time of GVI with sparse precision matrix is approximately 263.45s. Since the model here is more complicated than the models in Sections 4.2.1 and 4.2.2, for estimating the covariance matrix with preliminary samples of the MCMC algorithms, we run the MALA for 85000 iterations, taking around 252.11s, comparable to the time spent by GVI, and we run HMC for 10^5 iterations, taking around 3404.53s.

The running time, efficiency (E), average acceptance rate (P_{jump}) and lag-1 autocorrelation (ρ_1) are shown for each algorithm in table 4-5. Each result is an average over 3 replicated runs.

Table 4-5 Comparison of four MCMC algorithms on the polypharmacy dataset under the 509-dimensional GLMM model.

Method	Time(s)	E	P_{jump}	ρ_1
MALA	3157.145	0.0012	0.5767	0.9956
MALA-GVI	3118.427	0.0296	0.5742	0.9793
HMC	14071.61	0.7006	0.9745	0.7002
HMC-GVI	10455.96	0.9584	0.9537	-0.0053

From the results we can see that for sampling from high-dimensional and complex target distributions, deriving the transformation matrix in the MALA and HMC algorithms using GVI is beneficial. First, the running time of GVI is significantly shorter than that of the preliminary runs of MCMC. Furthermore, GVI yields a more accurate estimate of the covariance matrix and this improvement in accuracy leads to the increase in efficiency of the corresponding MCMC algorithms. Specifically, the efficiencies are 0.0296 and

0.9584 for the MALA and HMC respectively with transformation matrix estimated with GVI, while 0.0012 and 0.7006 respectively for the algorithms with transformation matrix estimated using the preliminary MCMC samples.

4.3 Efficient Bayesian Sampling Using the VAE-MH Algorithm

4.3.1 Sampling mixture of Gaussian and mixture of Student t

4.3.1.1 The model setup

Following the experiments in Habib et al. (2018)^[38], our target distributions are a mixture of two-dimensional Gaussian distributions and a mixture of two-dimensional Student t distributions with distant modes (a distance of 20 between means while the standard deviation is 1).

We formalize the expression of the mixture Gaussian and mixture Student t distributions as follows. Let $N(\boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1)$ and $N(\boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2)$ denote two Gaussian distributions with means $\boldsymbol{\mu}_1 = (10, 0)^\top$ and $\boldsymbol{\mu}_2 = (-10, 0)^\top$, and covariance matrices $\boldsymbol{\Sigma}_1 = \boldsymbol{\Sigma}_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ respectively. Then the mixture Gaussian distribution is

$$p(\mathbf{x})_{\text{Mix. of Gauss.}} = w \cdot N(\mathbf{x} \mid \boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1) + (1 - w) \cdot N(\mathbf{x} \mid \boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2),$$

where $0 < w < 1$ is the mixture weight.

Let $T(\boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1, \nu)$ and $T(\boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2, \nu)$ denote two Student t distributions with means $\boldsymbol{\mu}_1 = (10, 0)^\top$ and $\boldsymbol{\mu}_2 = (-10, 0)^\top$, and covariance matrices $\boldsymbol{\Sigma}_1 = \boldsymbol{\Sigma}_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ respectively. The degrees of freedom is $\nu = 5$ for both distributions. Then the mixture Student t distribution is

$$p(\mathbf{x})_{\text{Mix. of Student } t} = w \cdot T(\mathbf{x} \mid \boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1, 5) + (1 - w) \cdot T(\mathbf{x} \mid \boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2, 5).$$

In addition, we also consider the different degrees of freedom for the mixture of student t distribution, we set the $\nu_1 = 1, \nu_2 = 4$, then the target distribution for the mixture of student t becomes

$$p(\mathbf{x})_{\text{Mix. of Student } t} = w \cdot T(\mathbf{x} \mid \boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1, 1) + (1 - w) \cdot T(\mathbf{x} \mid \boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2, 4).$$

4.3.1.2 The simulation results

To sample from each of the mixture Gaussian and mixture Student t distributions, we compare the performance of the random-walk MH and VAE-MH algorithms, which are introduced in Section 3.1.1 and in Section 3.4.2 respectively. When sampling each target distribution, we run each algorithm for 10^5 iterations, and we show the corner plots of samples generated by the random-walk MH and VAE-MH samplers in figures 4-2, 4-3 and 4-4 respectively. The corner plot is an illustrative representation of the distributions of the samples for a high-dimensional variable. The corner plot has a lower-triangle shape, showing the marginal distribution of each component on the diagonal and the contour for each pair of components on the off-diagonal cells. Also, the running time and ESS per second for the two algorithms sampling the mixture Gaussian and mixture Student t are shown in tables 4-6, 4-7 and 4-8 respectively. Here we use ESS rather than E to facilitate the comparison with the results in Habib et al. (2018). Recall that ESS is the product of E and the total number of MCMC iterations, and thus the two quantities are equivalent.

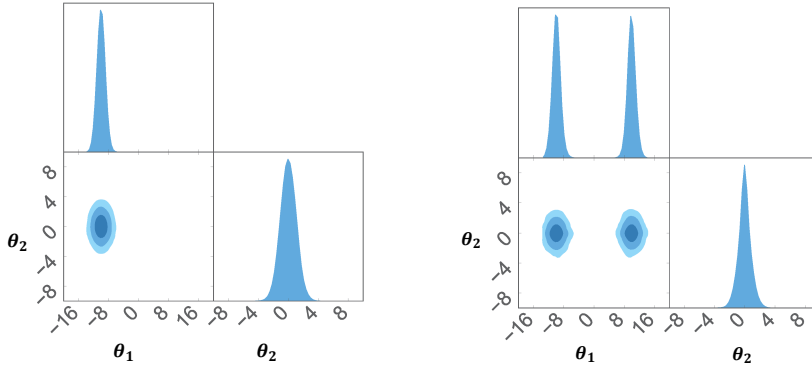


Figure 4-2 Corner plot of random-walk MH and VAE-MH algorithms for sampling the mixture Gaussian distribution.

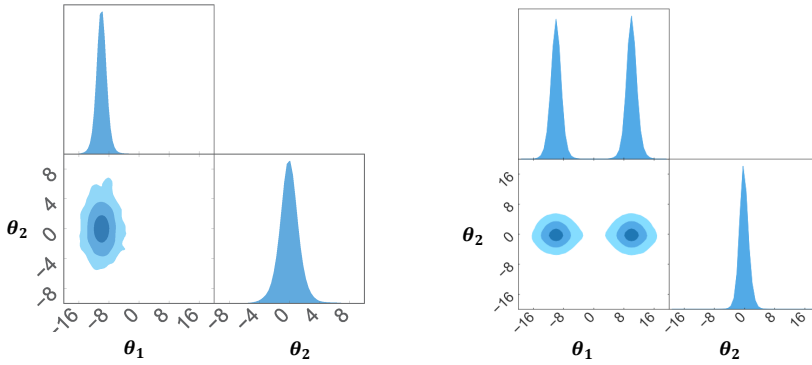


Figure 4-3 Corner plot of random-walk MH and VAE-MH algorithms for sampling the mixture Student t distributions with degrees of freedom $\nu_1 = \nu_2 = 5$.

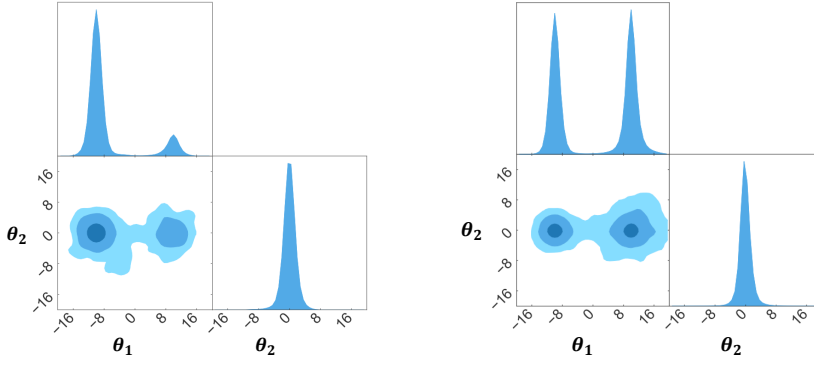


Figure 4-4 Corner plot of random-walk MH and VAE-MH algorithms for sampling the mixture Student t distributions with degrees of freedom $v_1 = 1, v_2 = 4$.

As illustrated in both figures 4-2 and 4-3, the random-walk MH algorithm gets stuck in one mode, while the VAE-MH sampler successfully explore the whole parameter space and find both modes which are in fact far away apart. In addition, in the mixture of student t distribution with different degrees of freedom $v_1 = 1, v_2 = 4$, the samples of RWM algorithm mainly concentrate in specific areas, indicating localized exploration with limited coverage of the distribution's potential multi-modality, while the VAE-MH algorithm demonstrates better exploration of the distribution's multi-modal nature, as evidenced by the clear multi-modal patterns in the joint distribution plots. Furthermore, figure 4-3 shows that the VAE-MH algorithm also captures the heavy-tailed feature of the Student t distribution better than random-walk MH. Since the random-walk MH gets stuck in one local mode and fails to traverse the whole parameter space, it can hardly converge. Therefore here we omit the ESS for the random-walk MH in the mixture of Gaussian experiment and mixture of student t with degrees of freedom $v_1 = v_2 = 5$, because estimating efficiency using a chain that does not converge is not sensible.

Table 4-6 Performance comparison of the random-walk MH and VAE-MH algorithms for sampling the mixture Gaussian distribution.

Algorithm	Time(s)	ESS/s
random-walk MH	145	NA
VAE-MH	288	12.9683

Table 4-7 Performance comparison of the random-walk MH and VAE-MH algorithms for sampling the mixture Student t distribution with degrees of freedom $v_1 = v_2 = 5$.

Algorithm	Time(s)	ESS/s
random-walk MH	122	NA
VAE-MH	266	3.7852

Table 4-8 Performance comparison of the random-walk MH and VAE-MH algorithms for sampling the mixture Student t distribution with degrees of freedom $v_1 = 1, v_2 = 4$.

Algorithm	Time(s)	ESS/s
random-walk MH	119.0788	5.720162
VAE-MH	309.0	17.2557

Table 4-9 ESS per second for each algorithm sampling the mixture Gaussian and mixture Student t distributions, taking into account the total time, including both the training time and generating time for 20000 samples

Target	AVS	HMC	RWM	ANICE	L2HMC
Mix. of Gauss.	7.64e-2	NA	NA	2.50e-4	2.88e-5
Mix. of Student t	1.45e-2	NA	NA	2.17e-4	3.7852

Furthermore, Habib et al. (2018)^[38] shows that it is not just difficult for basic MCMC samplers such as the random-walk MH to sample complex distributions efficiently, it is also difficult for more advanced MCMC algorithms such as HMC. L2HMC^[50] and ANICE-MCMC^[51] are both methods combining MCMC with neural networks. Habib et al. (2018)^[38] compared them with the method proposed by themselves, i.e., auxiliary variational MCMC (AVS), and two standard MCMC algorithms, which are random-walk MH and HMC. Habib et al. (2018) used the same mixture Gaussian and mixture Student t target distributions as in this section to do the comparison, and we copy their results in table 4-9. Here we can find that HMC also has trouble finding both modes for each of the mixture distributions. Furthermore, Habib et al. (2018) indicated that HMC also failed to capture the heavy-tailed shape of the Student t distribution.

4.3.2 Sampling from 10-dimensional mixture Gaussian distribution

4.3.2.1 The model setup

The third distribution that we aim to sample from is a mixture of two 10-dimensional Gaussian distributions. In fact, we use the same target distribution as deployed in the experiment of Hunt-Smith et al. (2024)^[40], and thus the density of the mixture Gaussian distribution is

$$f(\boldsymbol{\theta}) = w_A \frac{e^{-\frac{1}{2}\|\boldsymbol{\theta}-\boldsymbol{\theta}_A\|^2}}{(2\pi)^{d/2}} + w_B \frac{e^{-\frac{1}{2}\|\boldsymbol{\theta}-\boldsymbol{\theta}_B\|^2}}{(2\pi)^{d/2}},$$

where the weights are $w_A = \frac{2}{3}$, $w_B = \frac{1}{3}$ and the means are non-zero only for the first two dimensions, i.e., $\boldsymbol{\theta}_A = (8, 3, \mathbf{0}_8^\top)^\top$, $\boldsymbol{\theta}_B = (-2, 3, \mathbf{0}_8^\top)^\top$. This setup implies two distant modes in a 10-dimensional parameter space.

4.3.2.2 The simulation results

To sample from the 10-dimension mixture Gaussian distribution, we also compare the performance of random-walk MH and VAE-MH. For each algorithm, we run a chain of 10^5 iterations and generate the corresponding corner plot in figures 4-6 and 4-5 respectively. The total running time and ESS per second are shown in table 4-10.

From table 4-10, we can find that the running times of the VAE-MH and random-walk MH algorithms are comparable. However, the VAE-MH algorithm successfully identifies all the modes in the target distributions as shown in figure 4-6, while the random-walk MH gets trapped in one local mode as shown in figure 4-5 and fails to converge. Therefore, we do not display the ESS for the random-walk MH algorithm. For this more complicated 10-dimensional example, VAE-MH also shows its superiority in capacity of identifying multiple modes and efficiency of traversing the whole parameter space.

Table 4-10 Performance comparison of the random-walk MH and VAE-MH algorithms for sampling the 10-dimensional mixture Gaussian distribution.

Algorithm	Time(s)	ESS/s
random-walk MH	551	NA
VAE-MH	665	3.5536

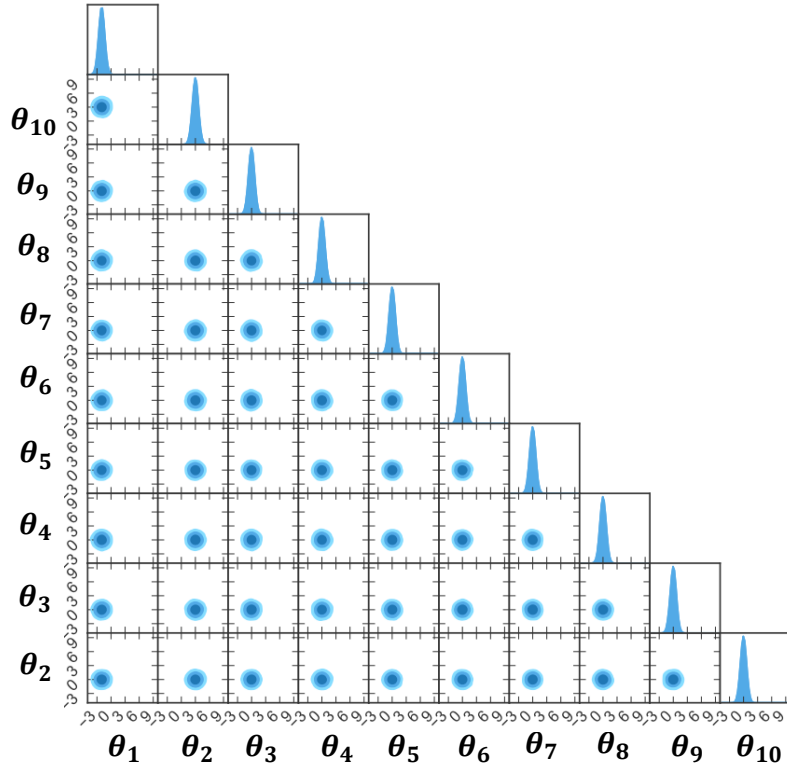


Figure 4-5 Corner plot of the random-walk MH algorithm for sampling the 10-dimensional mixture Gaussian distribution.

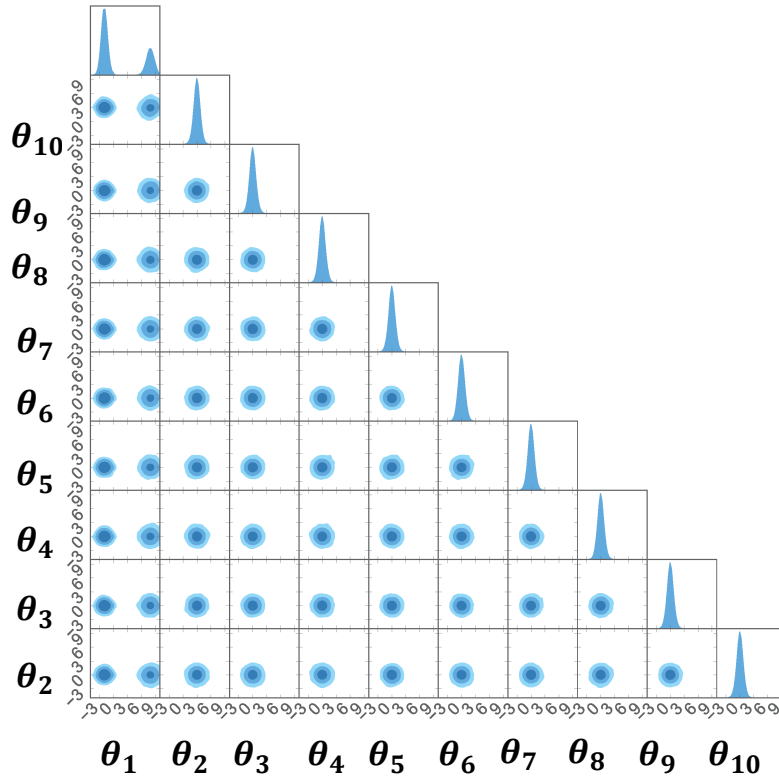


Figure 4-6 Corner plot of the VAE-MH algorithm for sampling the 10-dimensional mixture Gaussian distribution.

4.3.3 Sampling from 2-dimensional Gaussian mixture of 20 components

4.3.3.1 The model setup

The forth distribution that we aim to sample from is a mixture of 2-dimensional Gaussian mixtures with 20 components. In fact, we use the same target distribution as deployed in the experiment of Faming Liang et al. (2001)^[52], and thus the density of the mixture Gaussian distribution is

$$f(\boldsymbol{\theta}) = \frac{1}{\sqrt{2\pi}\sigma} \sum_{i=1}^{20} w_i \exp \left\{ -\frac{1}{2\sigma^2} (\boldsymbol{\theta} - \mu_i)^\top (\boldsymbol{\theta} - \mu_i) \right\},$$

where $\sigma = 0.1$ and $w_1 = \dots = w_{20} = 0.05$ and the mean vectors $\mu_1, \mu_1, \dots, \mu_{20}$ are uniformly drawn from the rectangle $[0, 10] \times [0, 10]$, these are listed in following table 4-11. And the variance of these modes are small enough to keep the spikes well separated, as might be inspected in figure 4-7. This setup implies 20 modes in a 2-dimensional parameter space.

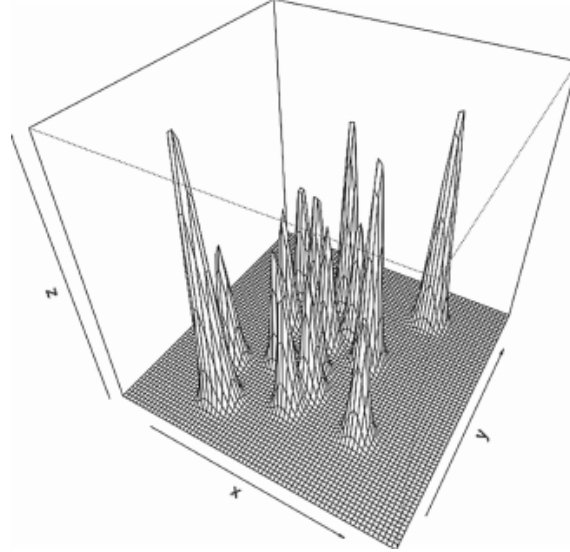


Figure 4-7 In a toy example involving twenty Gaussian peaks, it is evident that certain peaks cluster more densely, yet they are distinctly isolated by expansive areas where the probability is nearly zero.

Table 4-11 The Mean vectors of the 20 Components of the Mixture Gaussian Distribution.

i	μ_{i1}	μ_{i2}	i	μ_{i1}	μ_{i2}	i	μ_{i1}	μ_{i2}	i	μ_{i1}	μ_{i2}
1	2.18	5.76	6	3.25	3.47	11	5.41	2.65	16	4.93	1.50
2	8.67	9.59	7	1.70	0.5	12	2.70	7.88	17	1.83	0.09
3	4.24	8.48	8	4.59	5.60	13	4.98	3.70	18	2.26	0.31
4	8.41	1.68	9	6.91	5.81	14	1.14	2.39	19	5.54	6.86
5	3.93	8.82	10	6.87	5.40	15	8.33	9.50	20	1.69	8.11

4.3.3.2 The simulation results

To evaluate whether the VAE-MH algorithm can effectively sample from distributions with increased modality, we tested it on a two-dimensional mixture Gaussian distribution with 20 components. We conducted a simulation run comprising 10^6 iterations, including a burn-in period of 10^5 iterations. The simulation results are as follows.

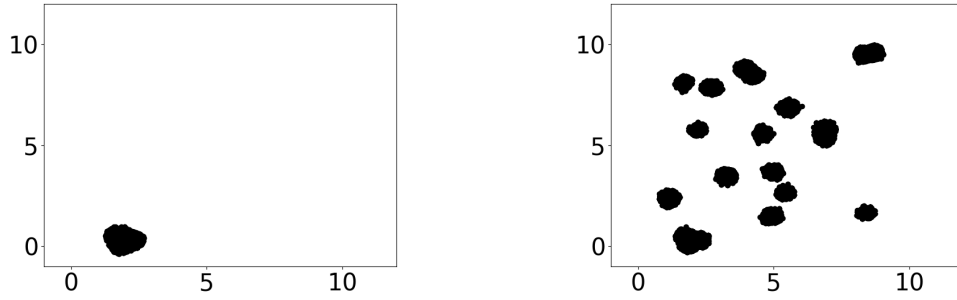


Figure 4-8 The plot of whole random-walk MH samples and VAE-MH samples.

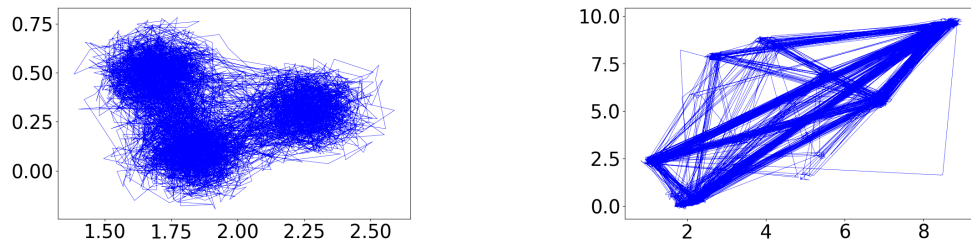


Figure 4-9 Trajectory plot of the first 10000 samples of random-walk MH and VAE-MH algorithm.

The figure 4-8 shows the plot of whole samples obtained via both the random-walk MH and VAE-MH algorithms. Furthermore, the figure 4-9 displays the sampling paths

for the first 10,000 iterations from both algorithms. This trace plot demonstrates that the VAE-MH algorithm successfully sampled all components within the initial 10,000 iterations, although the population was initialized at one corner. In contrast, in the first 10,000 iterations for the random-walk MH, the random-walk MH algorithm only detects the target distribution within the range $[0,3]$ and $[-0.5,1]$ and gets trapped in local modes. The plot of the whole random-walk MH samples shows that the random-walk MH sampler only detects the three modes of the target distribution.

CONCLUSION

1. Main Results

The goal of this study is to combine the MCMC algorithms with VI methods to produce a novel class of algorithms that incorporate the strengths of both. We propose two methods to combine MCMC and VI. The first is to use GVI to estimate the covariance matrix deployed by the HMC for linear transformation, and the second is to produce a new MH algorithm of which the proposal is produced by the VI-based generative model, VAE. By using the numerical studies to illustrate the effects of these two methods, we get the following three main conclusions.

(1) We find that moment estimates obtained from the burn-in stage of an MCMC algorithm which runs approximately the same amount of time as GVI are much less accurate. This is because the Markov chain is far from convergence at the end of burn-in, the distribution of these samples is biased and the problem becomes more severe as the dimension increases. GVI is much more computationally efficient in calculating the moment estimates of a distribution compared to MCMC and its estimation accuracy remains stable with the dimension of the target distribution increasing.

(2) Based on the experiment results of sampling from the logistic regression posterior distribution for the 8-dimensional Pima Indian dataset, the 24-dimensional German credit dataset, the 100-dimensional Gaussian target and 509-dimensional GLMM posterior distribution, distinct trends are observed in the comparison between GVI and MCMC in terms of the computational efficiency and the accuracy of estimating the covariance matrix. In low-dimension cases, the GVI and MCMC burn-in of 10^4 iterations take around the same amount of time. But GVI provides a more accurate covariance matrix estimate than MCMC burn-in, resulting in enhanced efficiency (E) of the MCMC algorithms for the transformed variables. In high-dimensional settings, GVI takes significantly shorter time than MCMC burn-in, while still provides a more accurate covariance matrix estimate. Thus estimating the covariance matrix with GVI leads to higher efficiency (E) of the HMC algorithm with linear transformation and shorter total sampling time. In all, GVI emerges as a promising alternative for the MCMC burn-in when estimating the covariance matrix, since it provides superior efficiency and accuracy, and its effects are robust for high-dimensional and complex target distributions.

(3) The new VAE-MH sampler that we proposed leverages the VAE model to capture the low-dimensional latent structures of the parameter space and generates a proposal distribution with global nature. Our experiments demonstrate that the new “VAE-MH” sampler performs better much than state-of-the-art MCMC algorithms for sampling multi-modal target distributions in terms of both efficiency and computational time.

2. Innovations

There are two main innovative points in this study.

(1) We innovatively propose to utilize GVI to derive a covariance matrix for the linear transformation in HMC. What is more, we use GVI with various covariance structures. Specifically, we consider GVI with Cholesky decomposed covariance^[12], GVI with factor decomposed covariance^[13] and GVI with sparse precision matrices^[14]. These methods address the challenge of estimating covariance matrices for complex and high-dimensional distributions.

(2) We propose to combine a VI-based generative model, VAE, with the random-walk MH sampler, by using the samples generated with the random-walk MH to train VAE and then constructing a new independent MH algorithm with the proposal produced by VAE. This innovative “VAE-MH” sampler is straightforward to implement and more efficient in traversing the whole parameter space than state-of-the-art MCMC algorithms.

3. Prospects

There are two directions that we can work on in the future.

(1) We have used some examples in our study to demonstrate the superiority of GVI over MCMC methods in estimating the covariance matrix and the efficiency of VAE-MH in sampling complex distributions compared to state-of-the-art MCMC algorithms. However, further investigation is necessary to fully explore their capabilities. In the future, we aim to assess their effectiveness of statistical inference in real application areas, such as finance, biology, etc.

(2) The VAE-MH sampler is a combination of one type of the generative models and one basic MCMC algorithm. In the future, we will investigate the possibility and effects of combining all sorts of generative models with more advanced MCMC algorithms, including the reversible jump MCMC^[53], which is a generalization of MH and allows jumping among parameter spaces with varying dimensions.

REFERENCES

- [1] METROPOLIS N, ROSENBLUTH A W, ROSENBLUTH M N, et al. Equation of state calculations by fast computing machines[J]. The Journal of Chemical Physics, 1953, 21(6): 1087-1092.
- [2] HASTINGS W K. Monte Carlo sampling methods using Markov chains and their applications [M]. Oxford University Press, 1970.
- [3] GIROLAMI M, CALDERHEAD B. Riemann manifold langevin and hamiltonian monte carlo methods[J]. Journal of the Royal Statistical Society Series B: Statistical Methodology, 2011, 73 (2): 123-214.
- [4] JORDAN M I, GHAHRAMANI Z, JAAKKOLA T S, et al. An introduction to variational methods for graphical models[J]. Machine Learning, 1999, 37: 183-233.
- [5] WAINWRIGHT M J, JORDAN M I, et al. Graphical models, exponential families, and variational inference[J]. Foundations and Trends® in Machine Learning, 2008, 1(1–2): 1-305.
- [6] BLEI D M, KUCUKELBIR A, MCAULIFFE J D. Variational inference: A review for statisticians[J]. Journal of the American Statistical Association, 2017, 112(518): 859-877.
- [7] ROBERT C P, CASELLA G. Monte Carlo statistical methods: Vol. 2[M]. Springer, 1999.
- [8] ZHANG C, MATSEN IV F A. Variational Bayesian phylogenetic inference[C]//International Conference on Learning Representations. 2018.
- [9] GAMERMAN D, LOPES H F. Markov chain Monte Carlo: stochastic simulation for Bayesian inference[M]. CRC press, 2006.
- [10] HABIB R, BARBER D. Auxiliary variational MCMC[C]//7th International Conference on Learning Representations, ICLR 2019. International Conference on Learning Representations, 2019: 1-13.
- [11] DE FREITAS N, HOJEN-SORENSEN P, JORDAN M I, et al. Variational mcmc[A]. 2013.
- [12] TRAN M N, NGUYEN T N, DAO V H. A practical tutorial on variational Bayes[A]. 2021.
- [13] ONG V M H, NOTT D J, SMITH M S. Gaussian variational approximation with a factor covariance structure[J]. Journal of Computational and Graphical Statistics, 2018, 27(3): 465-478.
- [14] TAN L S, NOTT D J. Gaussian variational approximation with sparse precision matrices[J]. Statistics and Computing, 2018, 28: 259-275.
- [15] KINGMA D P, WELLING M. Auto-encoding variational bayes[A]. 2013.
- [16] ROBERTS G O, ROSENTHAL J S. General state space Markov chains and MCMC algorithms [Z]. 2004.
- [17] METROPOLIS N, ULAM S. The monte carlo method[J]. Journal of the American Statistical Association, 1949, 44(247): 335-341.

REFERENCES

- [18] GEMAN S, GEMAN D. Stochastic relaxation, Gibbs distributions, and the Bayesian restoration of images[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 1984(6): 721-741.
- [19] YANG Z, RODRÍGUEZ C E. Searching for efficient Markov chain Monte Carlo proposal kernels[J]. Proceedings of the National Academy of Sciences, 2013, 110(48): 19307-19312.
- [20] THAWORNWATTANA Y, DALQUEN D, YANG Z. Designing simple and efficient Markov chain Monte Carlo proposal kernels[Z]. 2018.
- [21] ROBERTS G O, STRAMER O. Langevin diffusions and Metropolis-Hastings algorithms[J]. Methodology and Computing in Applied Probability, 2002, 4: 337-357.
- [22] NEAL R M, et al. MCMC using Hamiltonian dynamics[J]. Handbook of Markov Chain Monte Carlo, 2011, 2(11): 2.
- [23] HOFFMAN M D, GELMAN A, et al. The No-U-Turn sampler: adaptively setting path lengths in Hamiltonian Monte Carlo.[J]. J. Mach. Learn. Res., 2014, 15(1): 1593-1623.
- [24] HAARIO H, SAKSMAN E, TAMMINEN J. An adaptive Metropolis algorithm[J]. Bernoulli, 2001: 223-242.
- [25] GEYER C J. Markov chain Monte Carlo maximum likelihood[M]. Interface Foundation of North America, 1991.
- [26] MARINARI E, PARISI G. Simulated tempering: a new monte carlo scheme[J]. Europhysics letters, 1992, 19(6): 451.
- [27] TAK H, MENG X L, VAN DYK D A. A repelling-attracting Metropolis algorithm for multimodality[J]. Journal of Computational and Graphical Statistics, 2018, 27(3): 479-490.
- [28] BISHOP C. Pattern recognition and machine learning[J]. Springer Google Schola, 2006, 2: 5-43.
- [29] MNIH A, GREGOR K. Neural variational inference and learning in belief networks[C]//International Conference on Machine Learning. PMLR, 2014: 1791-1799.
- [30] MNIH A, REZENDE D. Variational inference for monte carlo objectives[C]//International Conference on Machine Learning. PMLR, 2016: 2188-2196.
- [31] PAISLEY J, BLEI D, JORDAN M. Variational Bayesian inference with stochastic search[A]. 2012.
- [32] ZEILER M D. Adadelata: an adaptive learning rate method[A]. 2012.
- [33] CRESWELL A, WHITE T, DUMOULIN V, et al. Generative adversarial networks: An overview[J]. IEEE Signal Processing Magazine, 2018, 35(1): 53-65.
- [34] KOBYZEV I, PRINCE S J, BRUBAKER M A. Normalizing flows: An introduction and review of current methods[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2020, 43(11): 3964-3979.
- [35] SOHL-DICKSTEIN J, WEISS E, MAHESWARANATHAN N, et al. Deep unsupervised learning using nonequilibrium thermodynamics[C]//International Conference on Machine Learning. PMLR, 2015: 2256-2265.
- [36] HO J, JAIN A, ABBEEL P. Denoising diffusion probabilistic models[J]. Advances in Neural Information Processing Systems, 2020, 33: 6840-6851.

REFERENCES

- [37] SALIMANS T, KINGMA D, WELLING M. Markov chain monte carlo and variational inference: Bridging the gap[C]//International Conference on Machine Learning. PMLR, 2015: 1218-1226.
- [38] HABIB R, BARBER D. Auxiliary variational MCMC[C]//International Conference on Learning Representations. 2018.
- [39] GABRIÉ M, ROTSKOFF G M, VANDEN-EIJNDEN E. Efficient bayesian sampling using normalizing flows to assist markov chain monte carlo methods[A]. 2021.
- [40] HUNT-SMITH N, MELNITCHOUK W, RINGER F, et al. Accelerating Markov Chain Monte Carlo sampling with diffusion models[J]. Computer Physics Communications, 2024, 296: 109059.
- [41] GELMAN A, GILKS W R, ROBERTS G O. Weak convergence and optimal scaling of random walk metropolis algorithms[J]. The annals of applied probability, 1997, 7(1): 110-120.
- [42] ROBERTS G O, ROSENTHAL J S. Examples of adaptive MCMC[J]. Journal of Computational and Graphical Statistics, 2009, 18(2): 349-367.
- [43] ROBERTS G O, ROSENTHAL J S. Optimal scaling of discrete approximations to Langevin diffusions[J]. Journal of the Royal Statistical Society: Series B (Statistical Methodology), 1998, 60(1): 255-268.
- [44] PARK J. Sampling from multimodal distributions using tempered Hamiltonian transitions[A]. 2021.
- [45] BETANCOURT M. A conceptual introduction to Hamiltonian Monte Carlo[A]. 2017.
- [46] THOMAS S, TU W. Learning hamiltonian monte carlo in r[J]. The American Statistician, 2021, 75(4): 403-413.
- [47] TRAN M N, NGUYEN N, NOTT D, et al. Bayesian deep net GLM and GLMM[J]. Journal of Computational and Graphical Statistics, 2020, 29(1): 97-113.
- [48] BARBU A, ZHU S C, et al. Monte Carlo Methods: Vol. 35[M]. Springer, 2020.
- [49] GELMAN A, JAKULIN A, PITTAU M G, et al. A weakly informative default prior distribution for logistic and other regression models[Z]. 2008.
- [50] LEVY D, HOFFMAN M D, SOHL-DICKSTEIN J. Generalizing hamiltonian monte carlo with neural networks[A]. 2017.
- [51] SONG J, ZHAO S, ERMON S. A-nice-mc: Adversarial training for mcmc[J]. Advances in Neural Information Processing Systems, 2017, 30.
- [52] LIANG F, WONG W H. Real-parameter evolutionary monte carlo with applications to bayesian mixture models[J]. Journal of the American Statistical Association, 2001, 96(454): 653-666.
- [53] GREEN P J. Reversible jump Markov chain Monte Carlo computation and Bayesian model determination[J]. Biometrika, 1995, 82(4): 711-732.

ACKNOWLEDGEMENTS

First of all, I want to thank my supervisor, Assistant Professor Xiyun JIAO, for her patient guidance in my study and research. She was involved in every step of my thesis, from selecting the topic to conducting the experiments and revising the paper. She helped me modify my code, explained the algorithms to me in her spare time, and cared for me in my daily life. She was rigorous and diligent in her academic work, and sincere and humble in her personality. She is my role model for my future learning. I wish her all the best in her future work and life.

Next, I want to express my gratitude to my grandparents for their encouragement and support throughout my journey of growth. Every conversation with them brings warmth and touches my heart, especially being far away from home. Furthermore, I want to express my gratitude to my parents who support and encourage me to bravely pursue what I love, allowing me to stand on their shoulders and see a bigger world.

I want to show my appreciate for our research group. Thanks to my senior fellow students of our research group, especially my senior student Guan Nuo and Fang Jialu. My academic discussion with them has made my research progress smoothly.

At last, I would also like to thank the teachers and students of the Department of Statistics and Data Science for their warm help, and I hope I can do my best to help them in the future. I would like to express my sincere gratitude to Professor Guoliang Tian for patiently reviewing my thesis. His detailed explanations have greatly benefited me. Special thanks to my senior Wang Qianqian, classmate Wu Xiaotong and friend Wang Yongwei. Time has witnessed our days of hard study together and our friendship. May we help each other in our future. Thank all my friends who have helped me.

“Artists call the moment between the sun starting to set and the darkness covering the earth, the ‘golden hour’. This moment is incomparably short, yet incomparably long. Everything is immersed in the golden glow of the sunset, radiating a tenderness that no other moment can present. This moment makes people indulge involuntarily.”

I want to say: Thank you to everyone who has treated me with kindness and gentleness. Thank you to everyone who has accompanied me. Thank you for giving me, ‘the golden hour’.

RESUME AND ACADEMIC ACHIEVEMENTS

Resume

Yin pingping was born in Anqing, Anhui Province on October 12, 1999.

Yin pingping was admitted to South China Agricultural University in Department of Mathematics and Information Science, majoring in statistics, and graduated with a Bachelor of Science degree in June 2022.

Since September 2022, she has studied in the Department of statistics and data science of Southern University of Science and Technology.